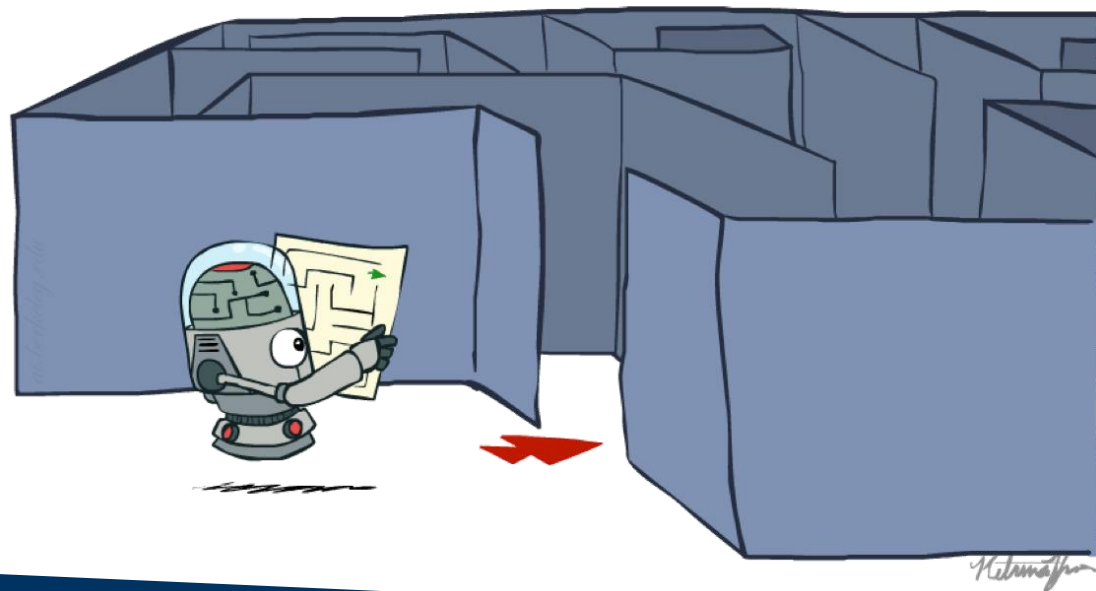
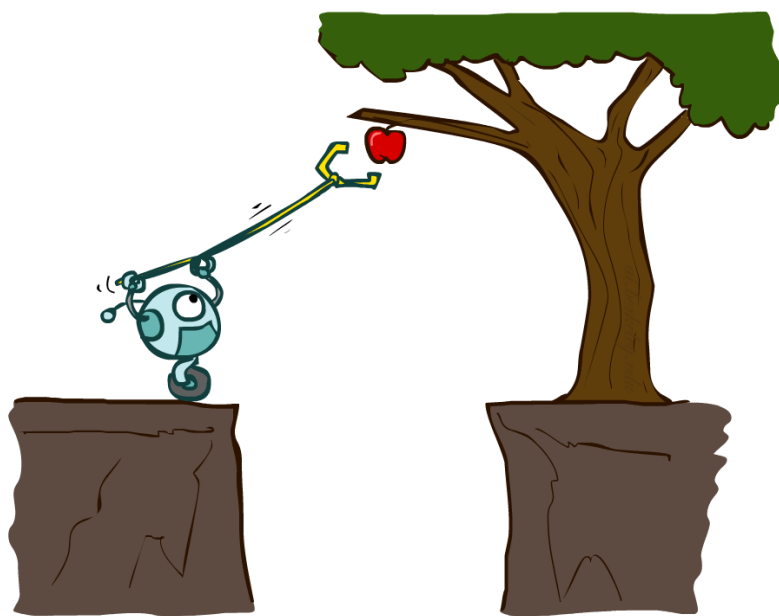


第二章 基础知识 搜索



智能体 与 环境

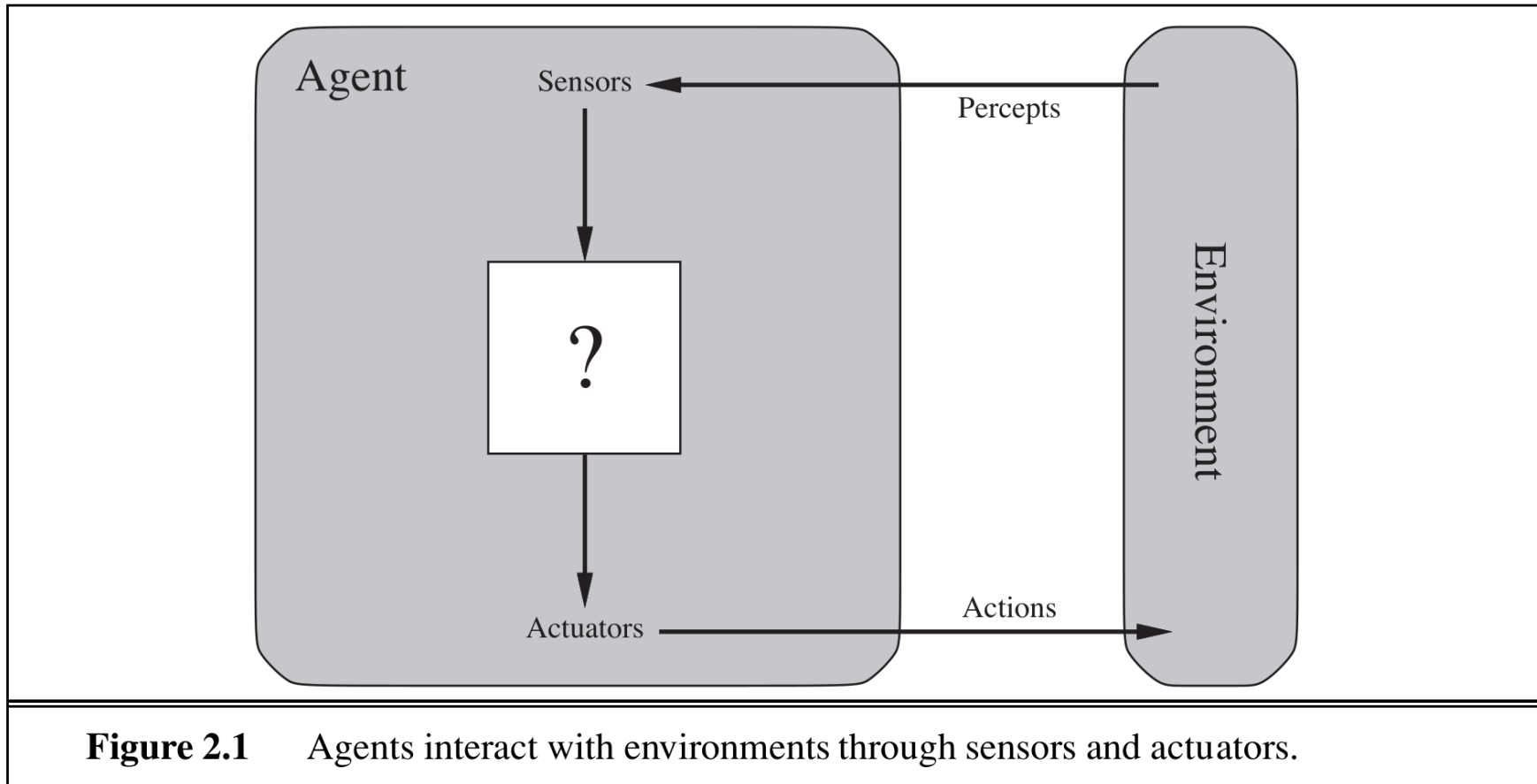


source: ai.berkeley.edu

补充内容：智能体

- 智能体（Agent, 人工智能领域中一个概念）：
 - 任何独立的能够思想并可以同环境交互的实体都可以抽象为智能体
 - 智能体/Agent：能自主活动的软件或硬件实体
 - 通过传感器/Sensors感知环境/Environment
 - 通过执行器/Actuator影响环境（执行动作/Action）

补充内容：智能体



补充内容：智能体

- 智能体的分类：

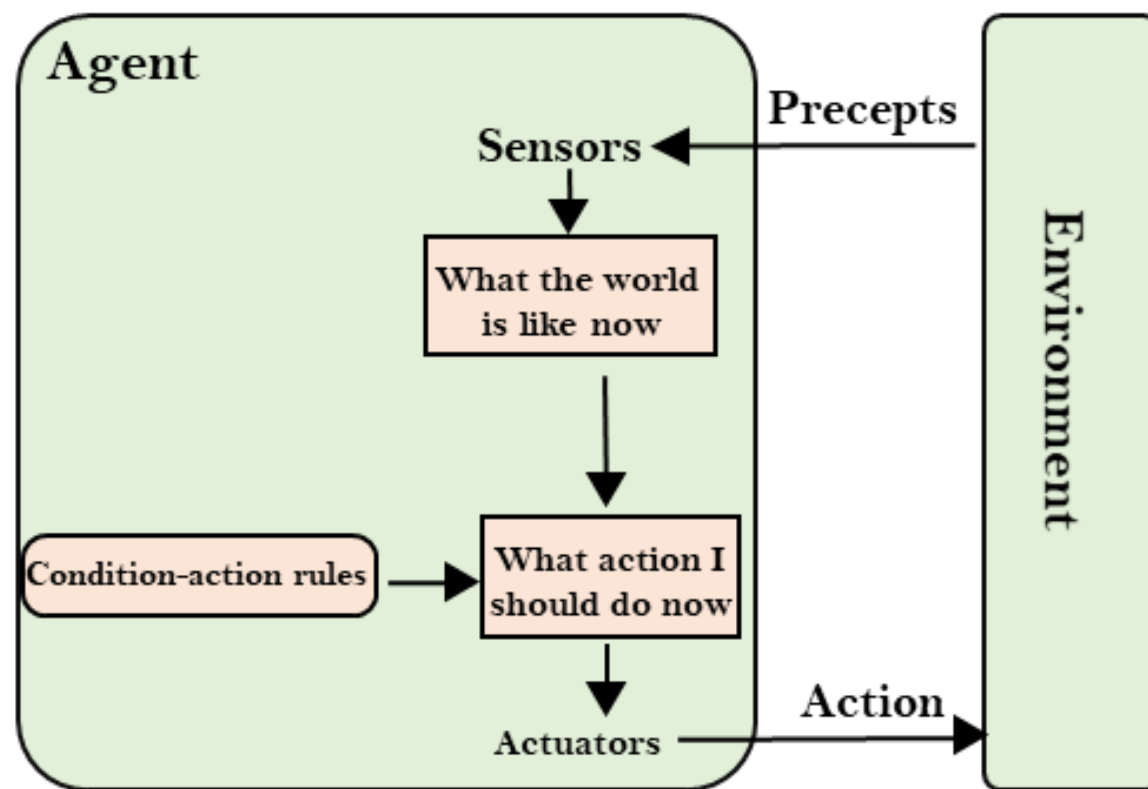
按照感知能力和处理方式分成五类：

- 简单反射式智能体 Simple Reflex Agent
- 模型反射式智能体 Model-based reflex agent
- 基于目标智能体 Goal-based agents
- 基于效用智能体 Utility-based agent
- 学习智能体 Learning agent

[Types of AI Agents - Javatpoint](#)

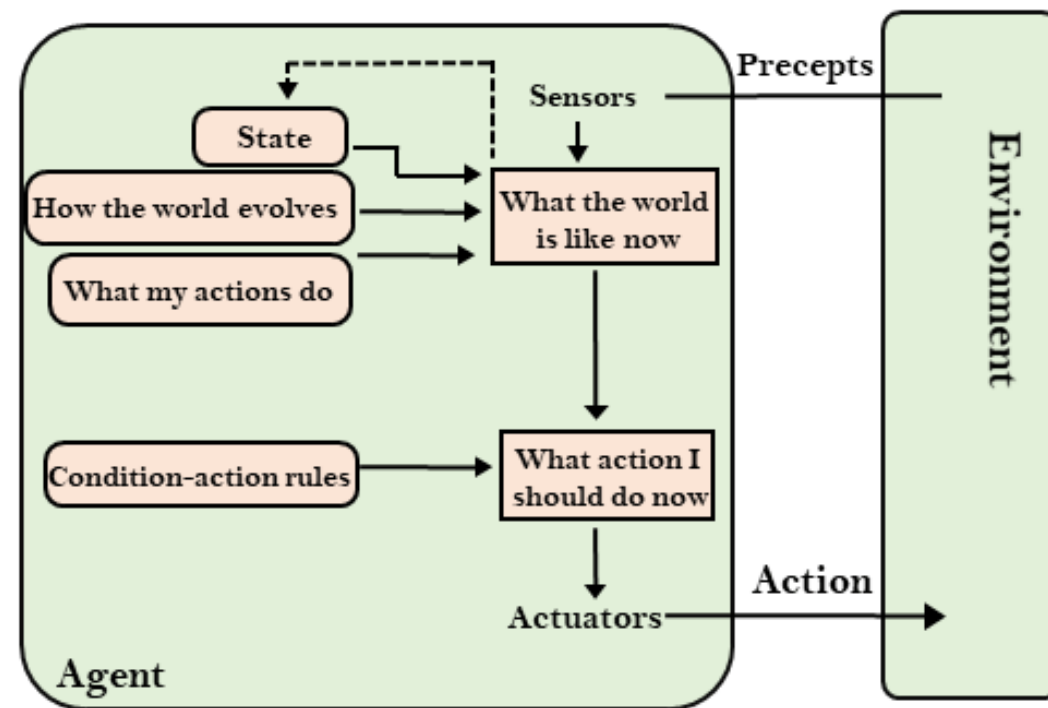
简单反射式智能体 Simple Reflex Agent

- 通过感知当前状态来决策
 - 忽略感知的历史
- 通过条件-动作规则来行动



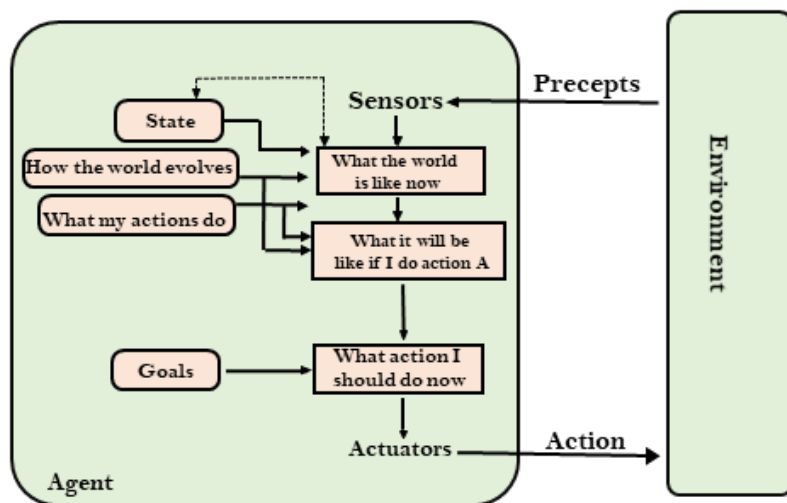
模型反射式智能体 Model-based reflex agent

- 相比简单反射智能体，拥有模型
 - 模型：描述对世界（环境）的认识（知识）
- 通过模型和传感器建立对世界的理解
 - 更复杂的条件（内部表达）



基于目标智能体 Goal-based agents

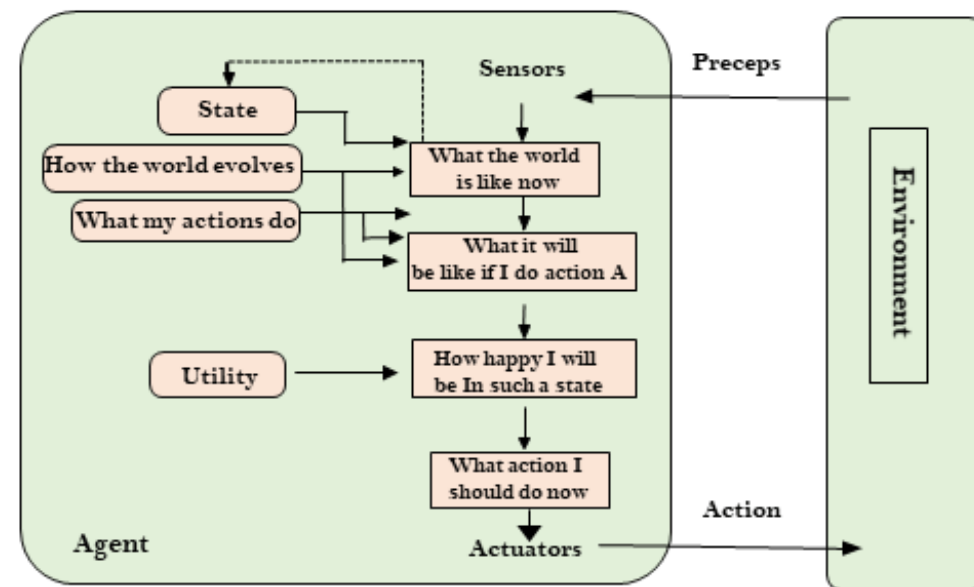
- 对当前环境的知识不够智能体决策，智能体需要了解动作的目标
- 相比模型反射智能体，增加了目标（终点）的信息
- 选择更能接近目标的动作



- 为了接近目标，智能体需要考虑一系列的可能动作来检验自己是否能达成目的
- 搜索与规划：考虑各种动作与相应场景的行为
- 搜索与规划使智能体的行动具有预见性 (Makes an agent proactive)

基于效用智能体 Utility-based agent

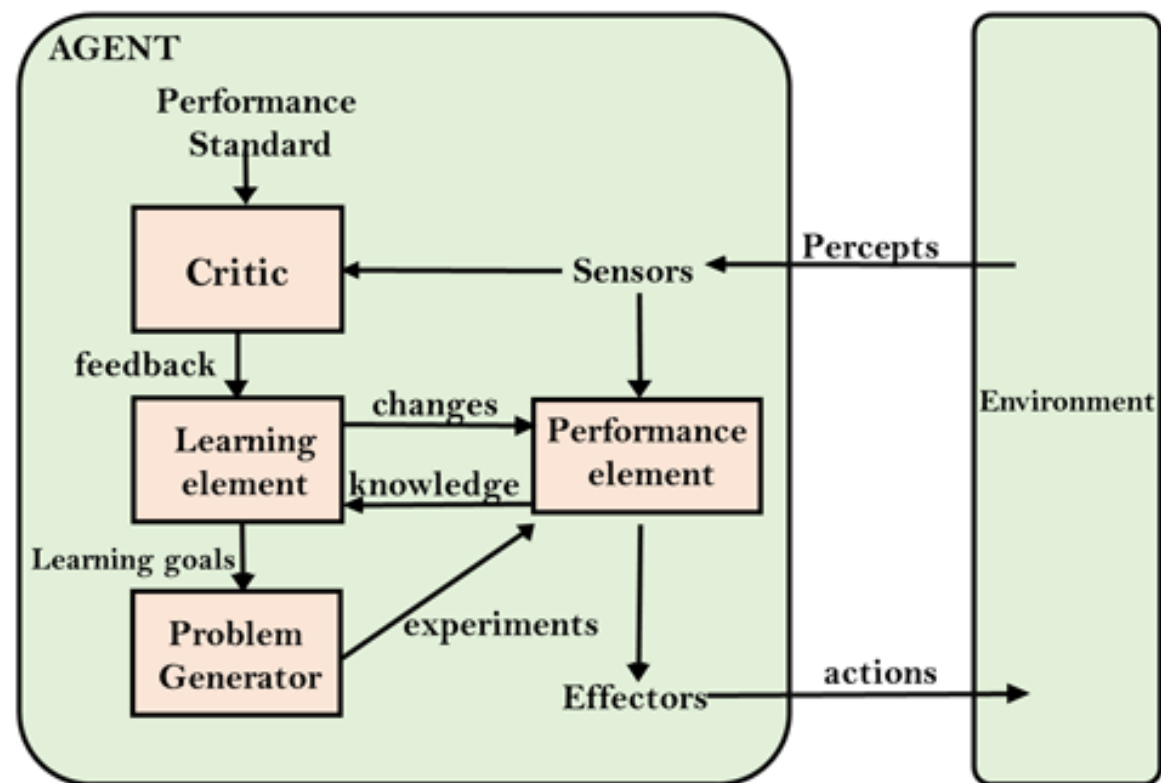
- 相比基于目标的智能体，基于增加了效用函数，从而能够评价当前状态有多**成功**
- 智能体的行动既追求达到目标，又追求以最佳方式达成目标
 - 效用能帮助智能体评价达到目标方式的好坏
 - 效用是数值化的评价



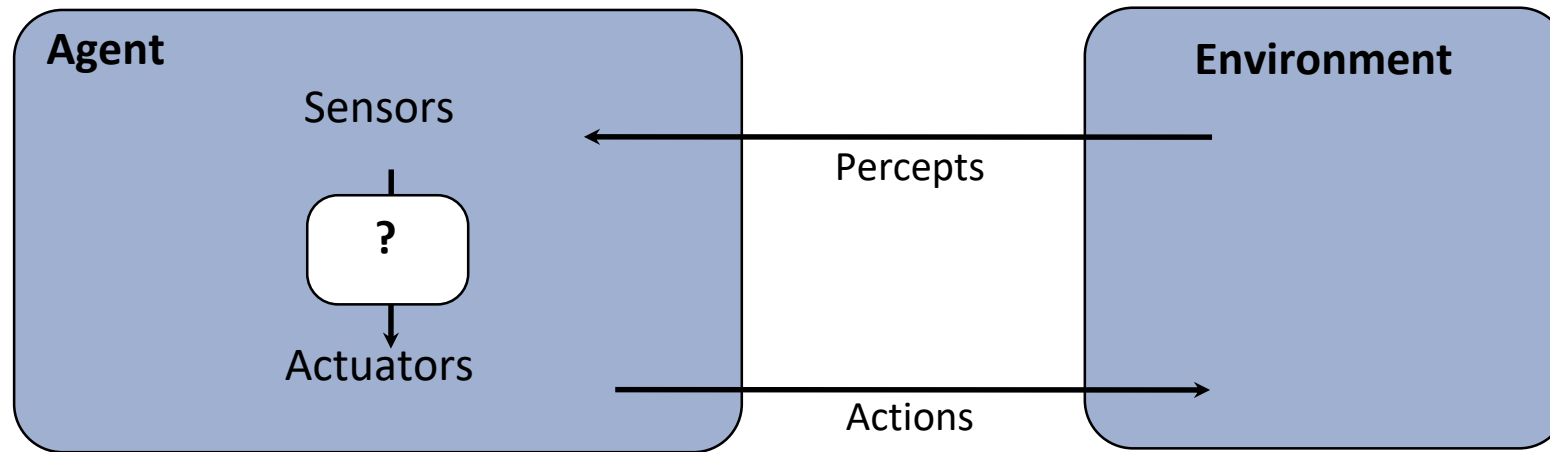
学习智能体 Learning agent

- 具有学习能力，能从过去经验中学习
- 从基本知识出发，通过学习来自适应
- 学习智能体的组成要件：

- a. 学习元素
 - Learning element
- b. 评价
 - Critic
- c. 性能元素
 - Performance element
- d. 问题生成
 - Problem generator

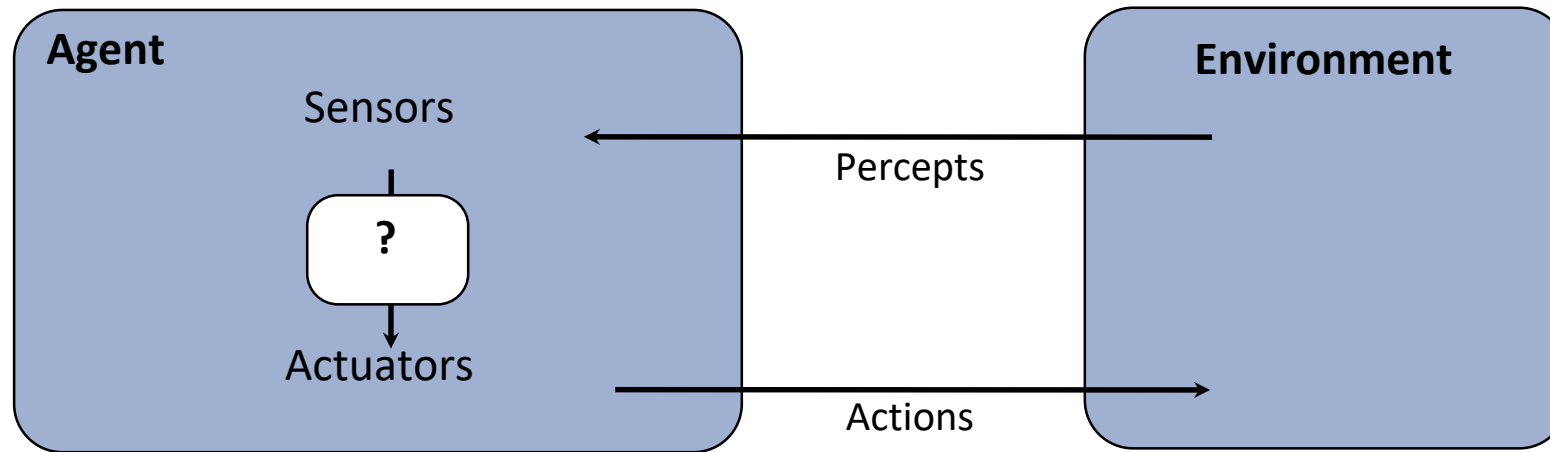


智能体与环境



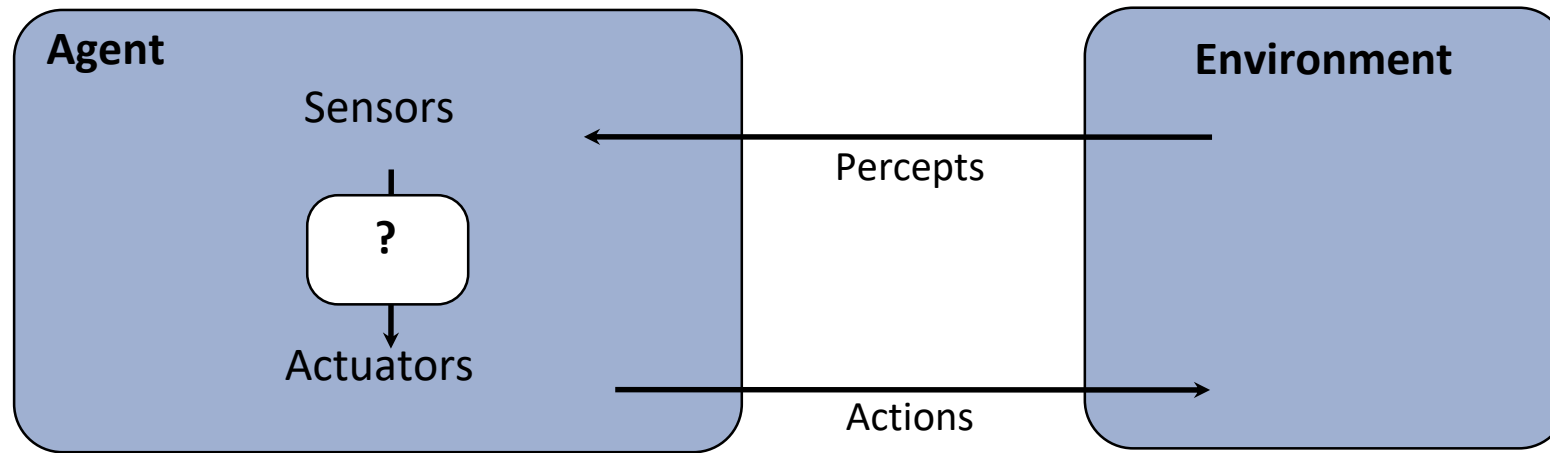
- 人类是智能体吗?
- Yes!
 - 传感器 = 视觉, 听觉, 嗅觉, 味觉, 触觉.....
 - 执行器 = 肌肉, 内分泌, 大脑活动.....

智能体与环境



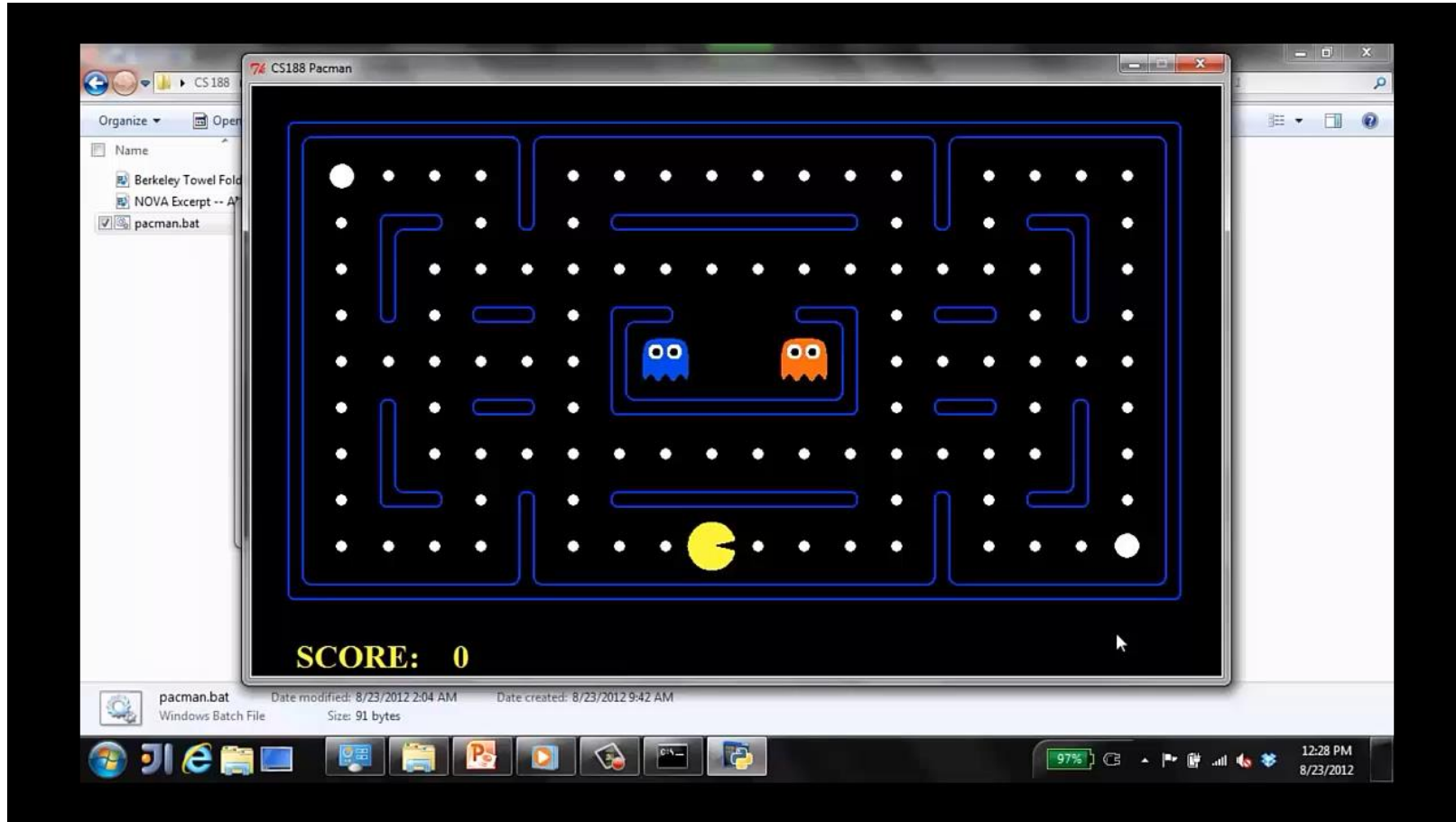
- 计算器是智能体吗?
- Yes!
 - 传感器 = 按键感应
 - 执行器 = 数字显示

智能体与环境



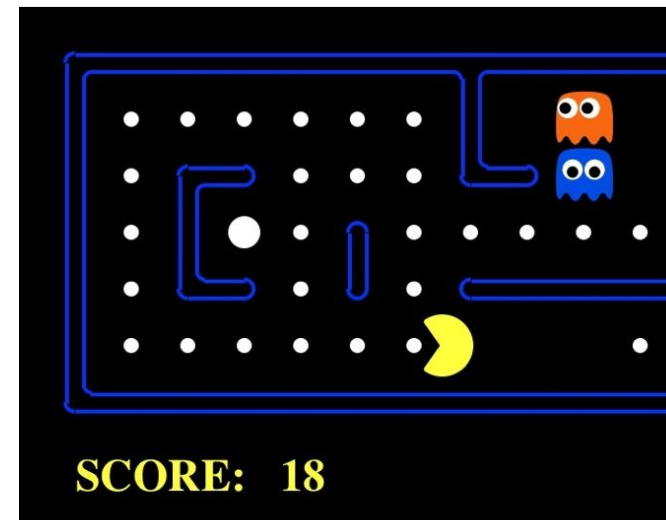
- 人工智能更关注：
 - 消耗庞大计算资源的智能体
 - 需要进行重要决策的环境

吃豆人中的类智能体

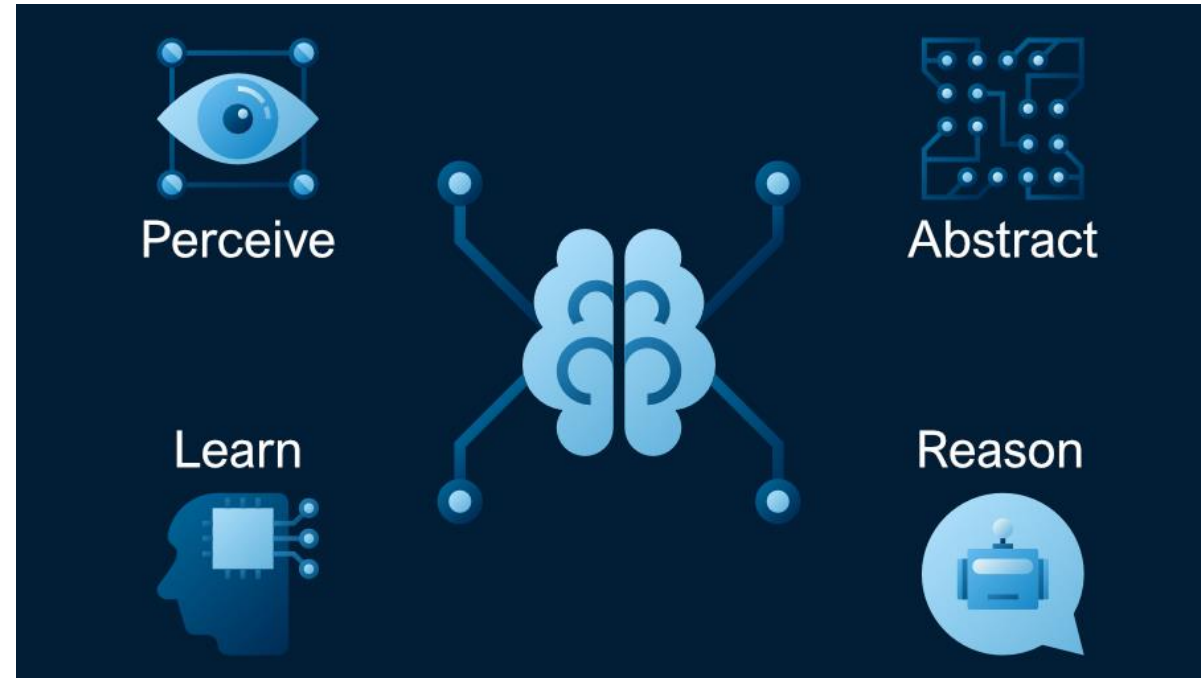


任务环境 - PEAS

- 性能度量
 - -1 per step; + 10 food; +500 win; -500 die; hit scared ghost
- 环境
 - 吃豆人游戏环境 (包括鬼魂行动)
- 执行器
 - 上下左右移动
- 传感器
 - 整个环境中状态可见 (除了无敌持续时间)



人工智能



Source: Six Kin Development

感知环境、求解问题、做出决策（包括计划、实时动作等）

计算机求解问题的方法

- 基于算法的方法：
 - 对问题本身的理解要求高
- 基于搜索的方法：
 - 对问题本身的理解要求较低
- 基于学习的方法：
 - 对问题本身的理解要求低

算法 vs. 搜索

- 例子：给定三角形ABC的顶点和某点 P 的坐标，判断点P是否在ABC内。
 - 算法求解：内角和法，同侧法，重心法
- 例子：计算平方根

快速算法

```

1 float InvSqrt(float x)
2 {
3     float xhalf = 0.5f*x;
4     int i = *(int*)&x;
5     i = 0x5f3759df - (i >> 1);
6     x = *(float*)&i;
7     x = x*(1.5f - xhalf*x*x);
8     return x;
9 }
    
```

二分法

以二分法求 $2 \sqrt{2}$

$\sqrt{\quad}$
 为例，1的平方等于1，2的平方等于4， $4 > 2$ ，因为函数单调递增，所以 $2 \sqrt{2}$
 $\sqrt{\quad}$
 的值肯定是在 (1, 2) 之间，然后取中间数1.5，而1.5的平方等于2.25， $2.25 > 2$ ，进一步缩小范围。
 $\sqrt{\quad}$
 的值肯定是在 (1, 1.5) 之间，再取中间数1.25，1.25的平方等于1.5625， $1.5625 < 2$ ，则 $2 \sqrt{2}$
 $\sqrt{\quad}$
 的值肯定是在 (1.25, 1.5) 之间，依次类推就可以一步步的逼近真实值。

搜索求解策略

- 在求解一个问题时：
 - 问题的表示，找到一个合适的表示方法
 - 选择一种相对合适的求解方法。由于绝大多数需要人工智能方法求解的问题缺乏直接求解的方法，因此，**搜索不失为一种求解问题的一般方法。** Search is useful for agents that plan ahead!
- 本章概要：
 - 搜索的基本概念
 - 状态空间知识表示和搜索策略
 - 盲目搜索
 - 启发式搜索

本章主要内容

- 搜索的基本概念
- 状态空间表示方法
- 盲目搜索策略
- 启发式搜索策略

搜索的基本问题与主要过程

- 搜索中需要解决的基本问题：
 - (1) 是否一定能找到一个解 Completeness
 - (2) 找到的解是否是最佳解 Optimality
 - (3) 时间与空间复杂性如何 Time & space complexity
 - (4) 是否终止运行或是否会陷入一个死循环 Goal test
- 搜索的主要过程：
 - (1) 从初始或目的状态出发，并将它作为当前状态。
 - (2) 扫描操作算子集，将适用当前状态的一些操作算子作用于当前状态而得到新的状态，并建立指向其父结点的指针。
 - (3) 检查所生成的新状态是否满足结束状态，如果满足，则得到问题的一个解，并可沿着有关指针从结束状态反向到达开始状态，给出一解答路径；否则，将新状态作为当前状态，返回第(2)步再进行搜索。

搜索方向

(1) 数据驱动：从初始状态出发的正向搜索

- 正向搜索——从问题给出的条件出发。

(2) 目的驱动：从目的状态出发的逆向搜索

- 逆向搜索——从想达到的目的入手，看哪些操作算子能产生该目的以及应用这些操作算子产生目的时需要哪些条件。

(3) 双向搜索

- 双向搜索——从开始状态出发正向搜索，同时又从目的状态出发逆向搜索，直到两条路径在中间的某处汇合为止。

搜索策略

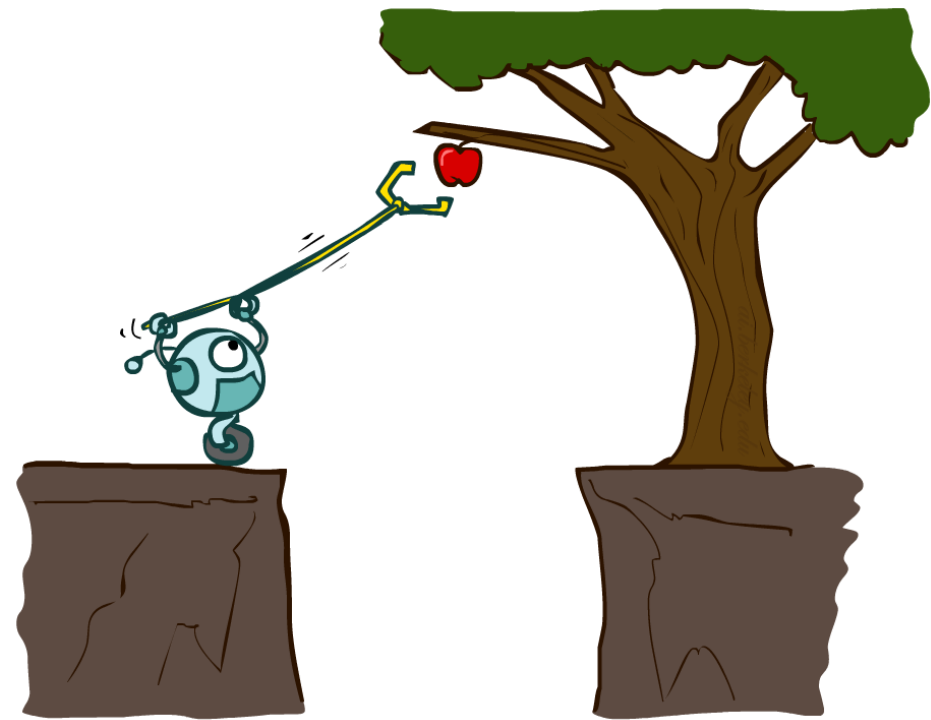
- (1) **盲目搜索**：在不具有对特定问题的任何有关信息的条件下，按固定的步骤（依次或随机调用操作算子）进行的搜索。
- (2) **启发式搜索**：考虑特定问题领域可应用的知识，动态地确定调用操作算子的步骤，优先选择较适合的操作算子，尽量减少不必要的搜索，以求尽快地到达结束状态。

本章主要内容

- 搜索的基本概念
- 问题建模—状态空间表示方法
- 盲目搜索策略
- 启发式搜索策略

规划智能体

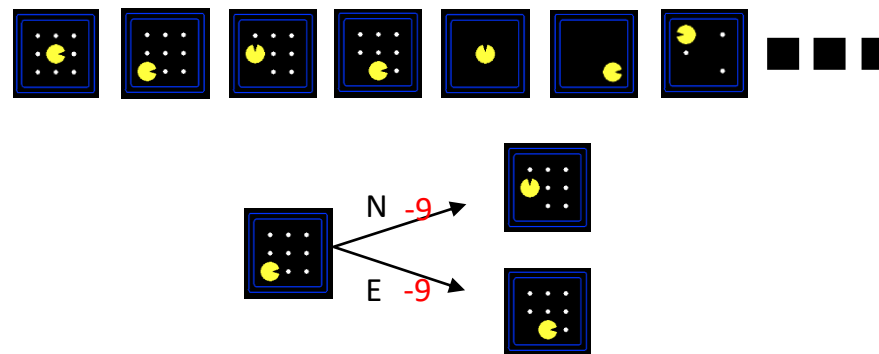
- Planning agents decide based on evaluating future action sequences
- Must have a model of how the world evolves in response to actions
- Usually have a definite goal
- Optimal: Achieve goal at least cost



Search Problems

• 一个搜索问题包含:

- 状态空间 S
- 初始状态 s_0
- 每个状态下的动作集合 $A(s)$
- 转移模型 $Result(s, a)$
- 目标验证 $G(s)$
 - 没有豆子剩下的 S
- 动作代价 $c(s, a, s')$
 - +1 per step; -10 food; -500 win; +500 die; -200 eat ghost

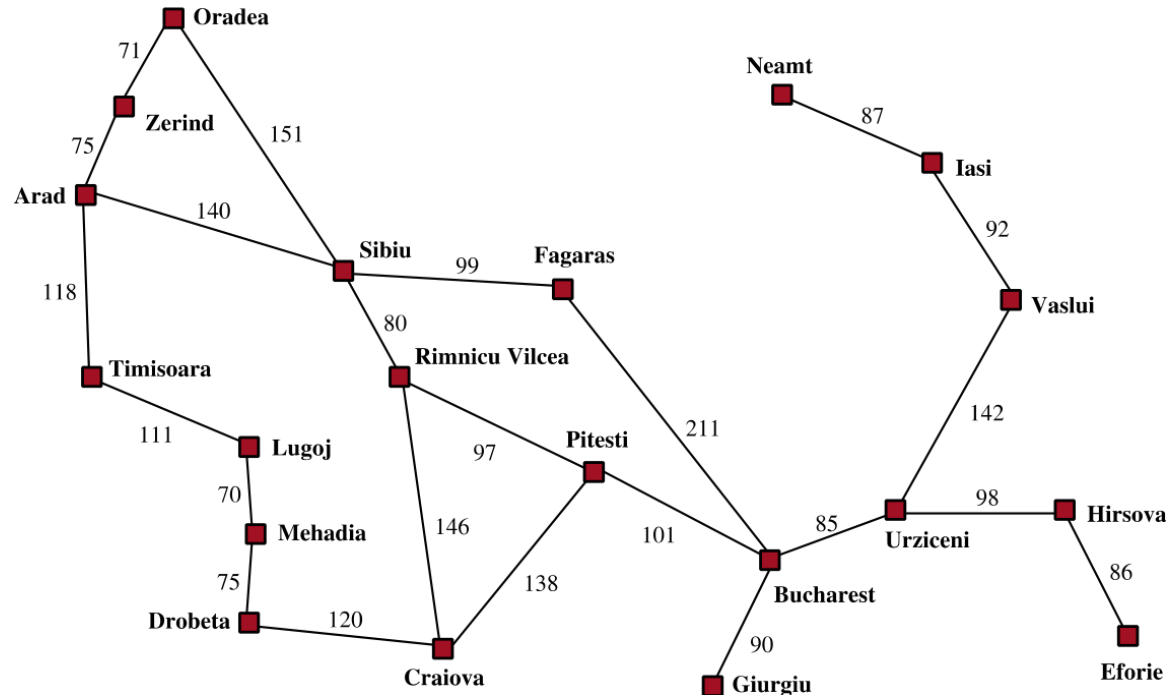


- 一个解是一个能达到目标状态的动作序列
- 最优解是所有解中最低的cost

Example: Traveling in Romania



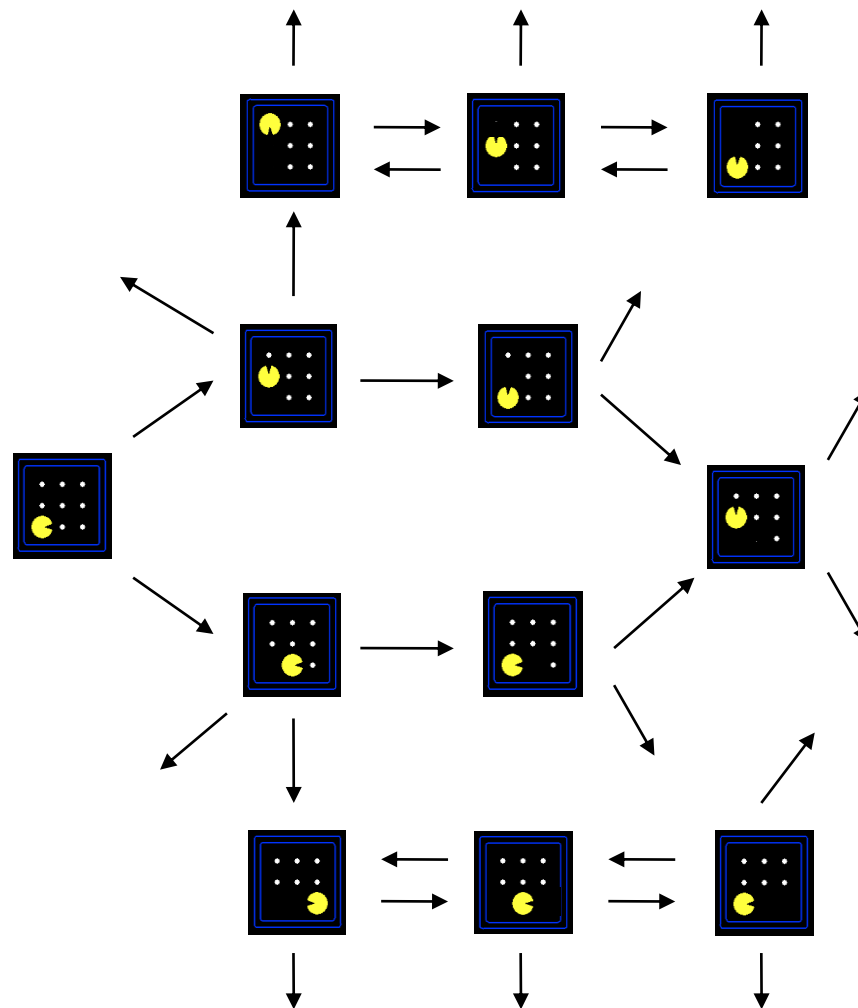
Example: Traveling in Romania



- 状态空间:
 - 访问的城市
- 初始状态
 - Arad
- 动作:
 - 旅行到相邻的城市
- 转移模型:
 - 到达相邻的城市
- 目标检验:
 - $s = \text{Bucharest?}$
- 动作代价:
 - 从 s to s' 的路径长度
- 解是什么?

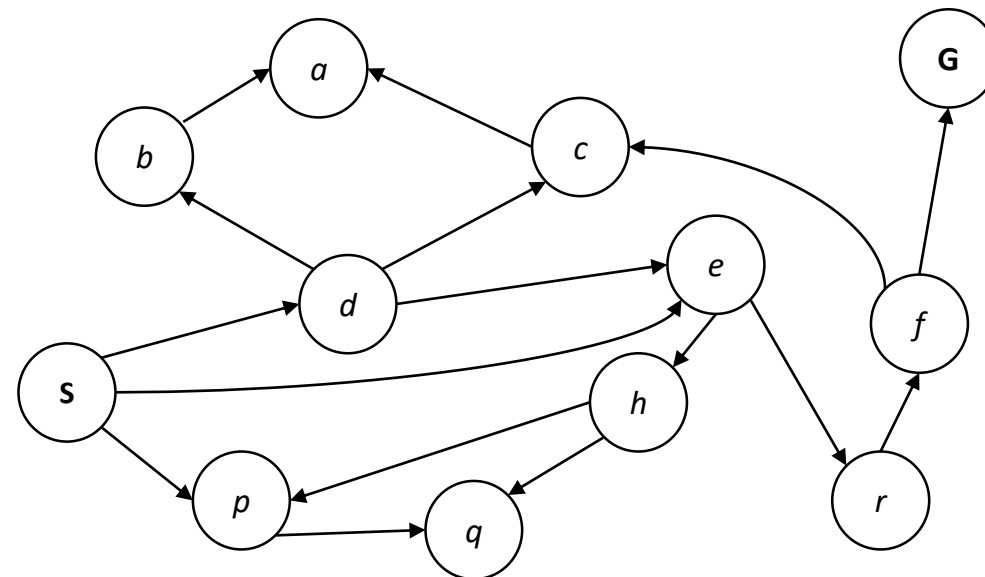
State Space Graphs

- 状态空间图: 搜索问题的数学表达
 - 节点 (抽象了) 环境的状态
 - 边表示状态转移 (标记出造成转移的动作)
 - 目标检验表示为目标节点 (可能唯一)
- 状态空间图中, 每个状态只出现一次
- 实际上很难完整画出状态空间图
 - 占用过多存储空间
 - 思想非常有用!



State Space Graphs

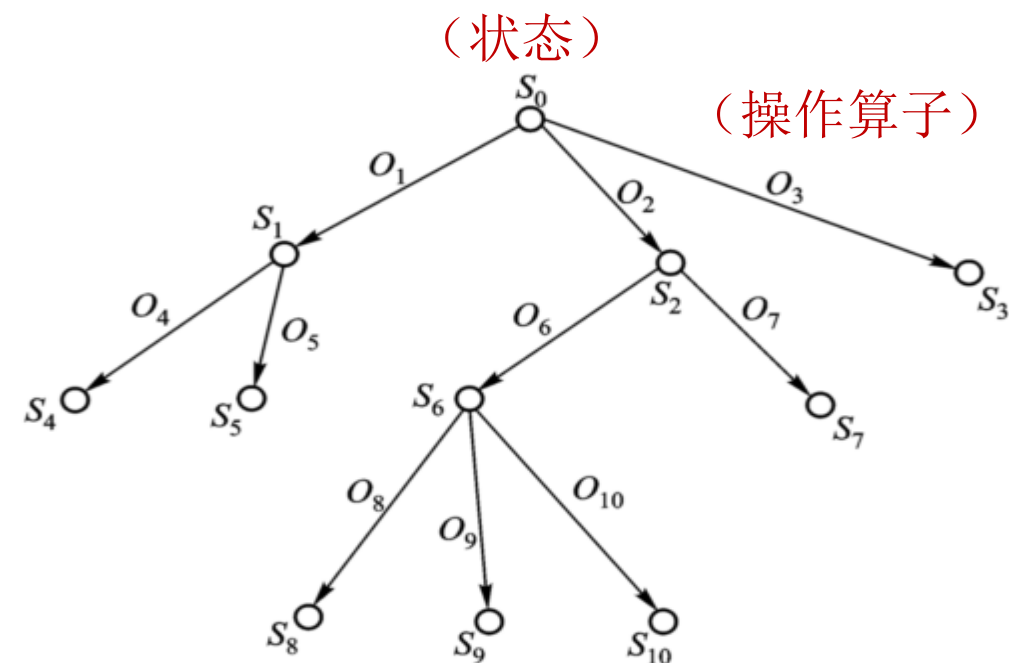
- 状态空间图: 搜索问题的数学表达
 - 节点 (抽象了) 环境的状态
 - 边表示状态转移 (标记出造成转移的动作)
 - 目标检验表示为目标节点 (可能唯一)
- 状态空间图中, 每个状态只出现一次
- 实际上很难完整画出状态空间图
 - 占用过多存储空间
 - 思想非常有用!



Tiny state space graph for a tiny search problem

搜索树

- 用有向图描述搜索过程
- 结点表示状态、边表示求解步骤
- 初始状态为根结点
- 一种状态转换为另一种状态的操作
算子序列 $\leftrightarrow$ 图中寻找某一路径



八数码问题的搜索树

- 例：八数码问题的状态空间

2	3	1
5		8
4	6	7

初始状态

1	2	3
8		4
7	6	5

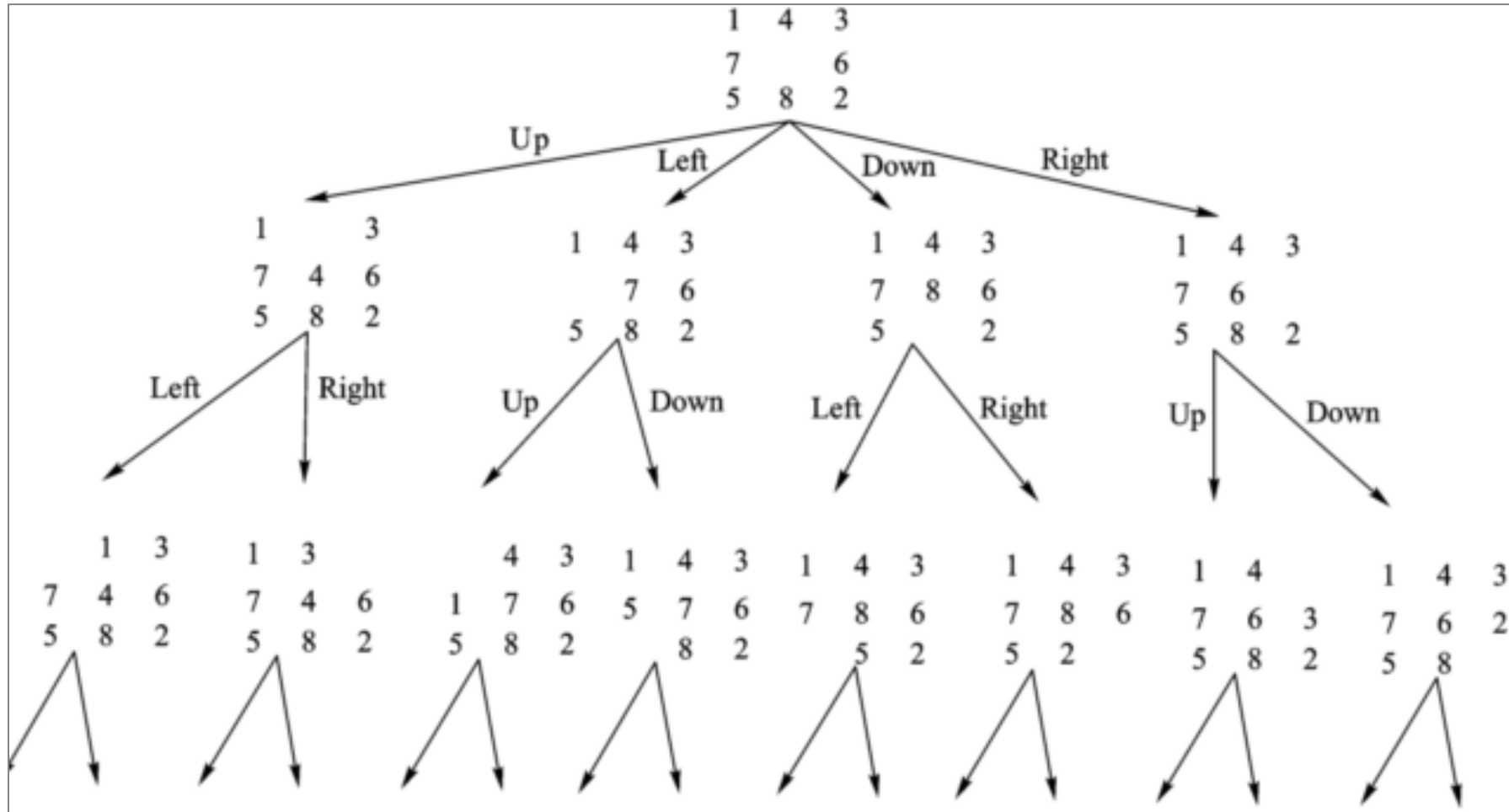
目标状态

状态集 S : 所有摆法

操作算子:

- 将空格向上移Up
- 将空格向左移Left
- 将空格向下移Down
- 将空格向右移Right

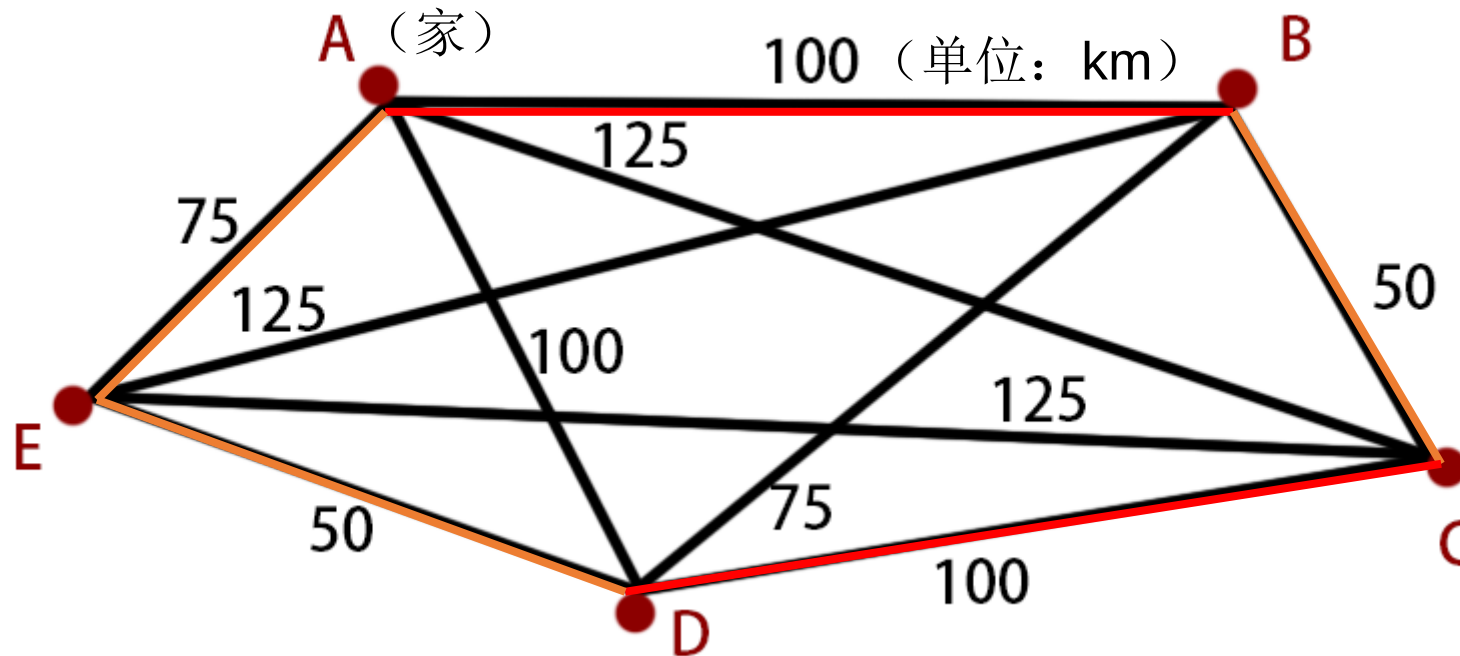
八数码问题的搜索树



八数码状态空间图

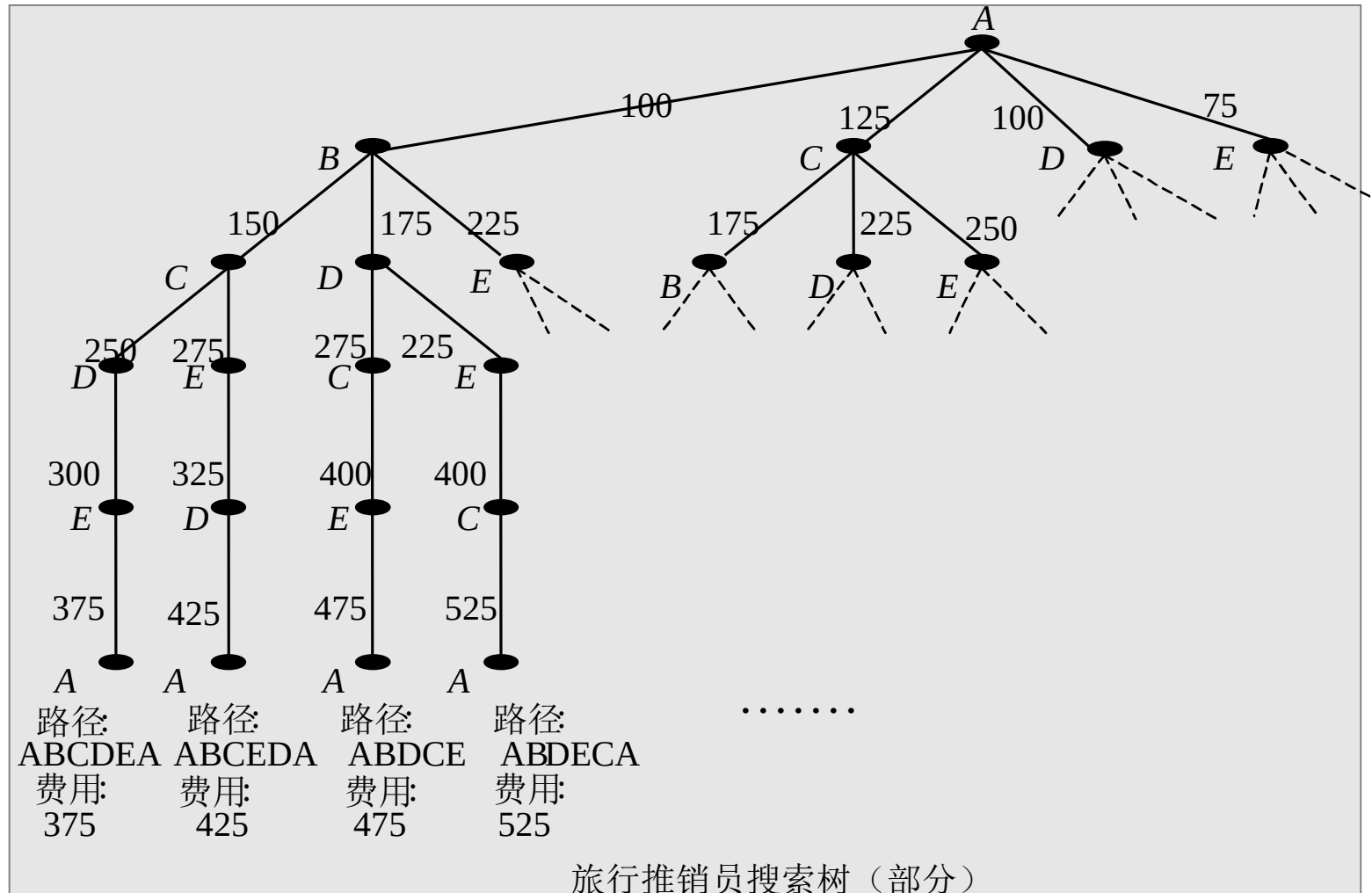
旅行商问题的搜索树

- 例：旅行商问题 (traveling salesman problem, TSP)

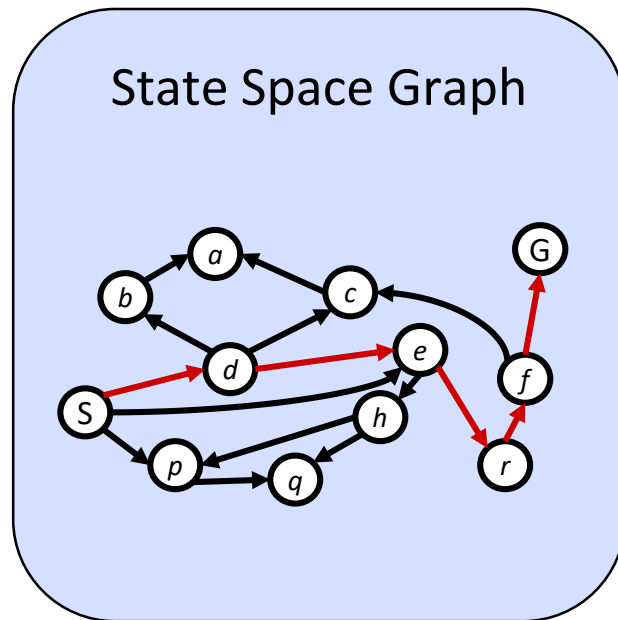


可能路径：费用为375的路径 (A, B, C, D, E, A)

旅行商问题的搜索树

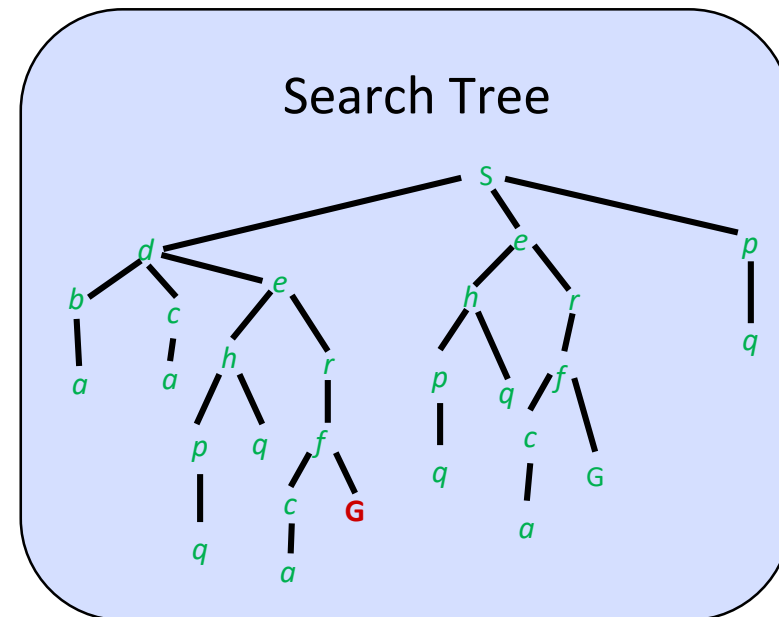


状态空间图 vs. 搜索树



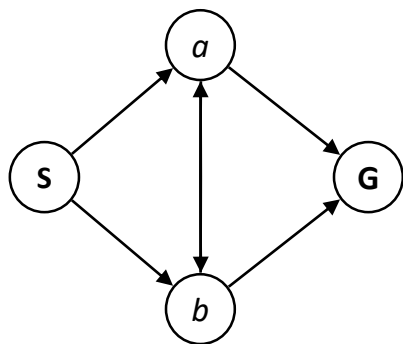
Each NODE in the search tree is an entire PATH in the state space graph.

We construct the tree on demand – and we construct as little as possible.



小测验：状态空间图 vs. 搜索树

考虑下面的四状态图：

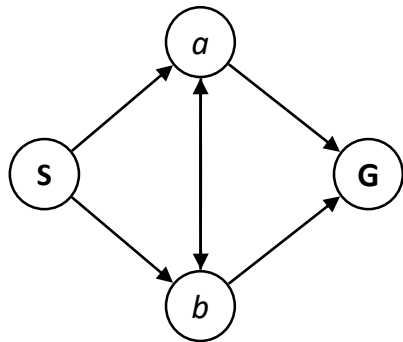


它的搜索树有多大 (以s为起点)?

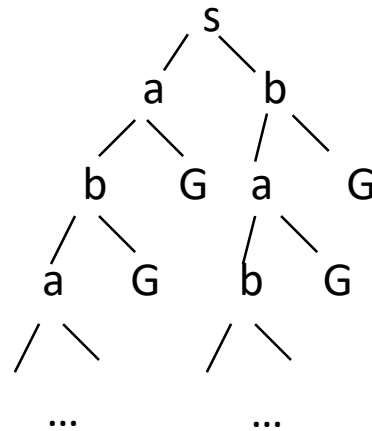


小测验：状态空间图 vs. 搜索树

考虑下面的四状态图：



它的搜索树有多大 (以s为起点)?



Important: Those who don't know history are doomed to repeat it!

本章主要内容

- 搜索的基本概念
- 状态空间表示方法
- 盲目搜索策略
- 启发式搜索策略

深度优先搜索

- **深度优先搜索** (Depth-first search, DFS) : 首先扩展最新产生的节点, 深度相等的节点按生成次序的盲目搜索。

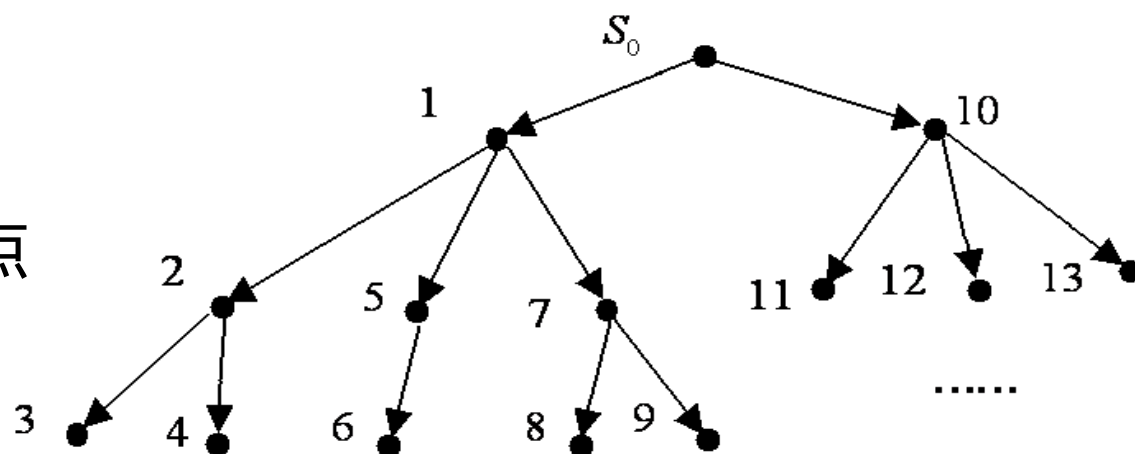


深度优先搜索

- **深度优先搜索** (Depth-first search) : 首先扩展最新产生的节点, 深度相等的节点按生成次序的**盲目**搜索。

特点:

- (1) 扩展最深的节点使得搜索沿着状态空间某条单一的路径从起始节点向下进行;
- (2) 仅当搜索到达一个没有后裔的状态时, 才考虑另一条替代的路径。



深度优先搜索

算法:

- 防止搜索过程沿着无益的路径扩展下去，往往给出一个节点扩展的最大深度——**深度界限**；
- 将扩展的后继节点放在OPEN表的前端；
- 深度优先搜索算法的OPEN表是堆栈，先进后出（FILO）。

深度优先搜索

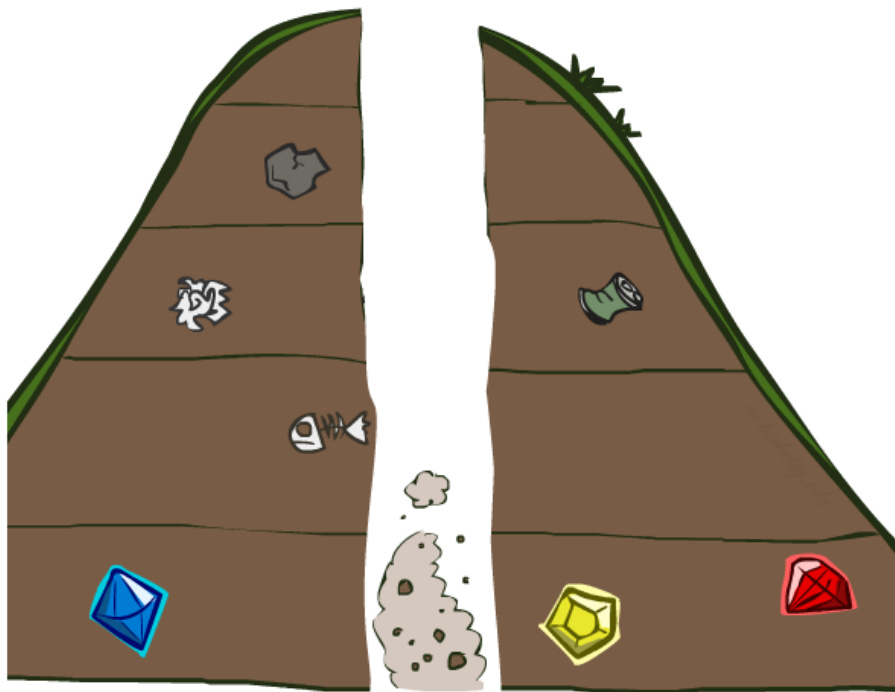
- 通常来讲，在Search的过程中，会将所有的点分成几个部分，Explored, Frontier, Unexplored
- Explored 是那些已经找到从起始点到该点的最佳路径的那些点
- Frontier 是还在寻找最优路径的那些点
- Unexplored是那些我们都还没有预见到的点。
- 搜索过程是：把Unexplored的点放到Frontier集合里面，然后把Frontier集合里的点挪到Explored集合里面。当最终的节点挪到Explored里面之后，整个搜索过程就结束了。

深度优先搜索

- 在深度优先搜索中，当搜索到某一个状态时，它所有的子状态以及子状态的后裔状态都必须先于该状态的兄弟状态被搜索。
- 为了保证找到解，应选择合适的深度限制值，或采取不断加大深度限制值的办法，反复搜索，直到找到解。
- 对任何状态而言，以后的搜索有可能找到另一条通向它的路径。如果路径的长度对解题很关键的话，当算法多次搜索到同一个状态时，它应该保留最短路径。

深度优先搜索

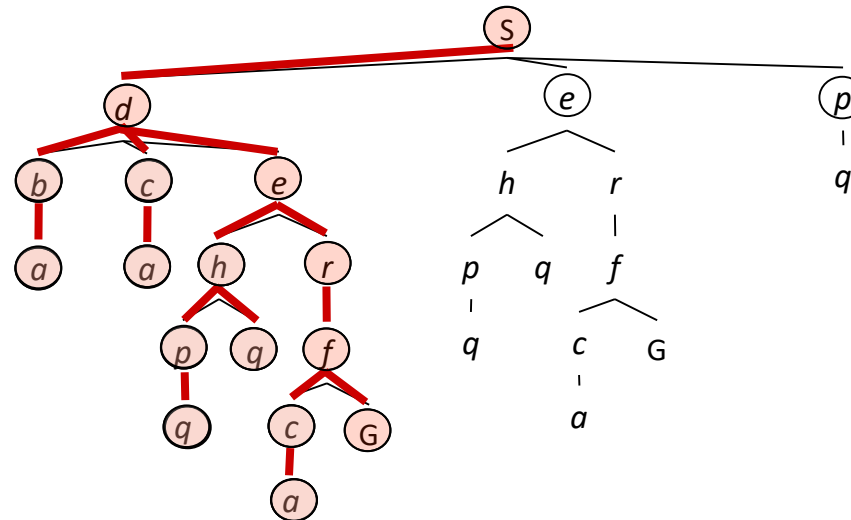
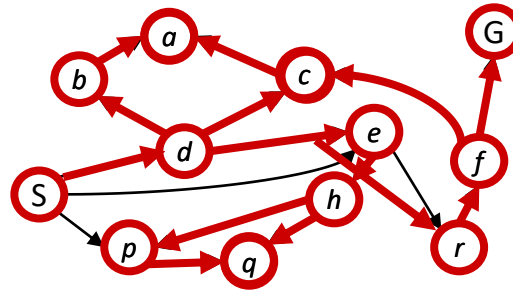
- 深度优先搜索**并不能保证第一次**搜索到的某个状态时的路径是到这个状态的**最短路径**。



Depth-First Search

Strategy: expand a deepest node first

Implementation: Frontier is a LIFO stack



宽度优先搜索

- **宽度优先搜索** (Breadth-first search, BFS)：以接近起始节点的程度（深度）为依据，进行逐层扩展的节点搜索方法。

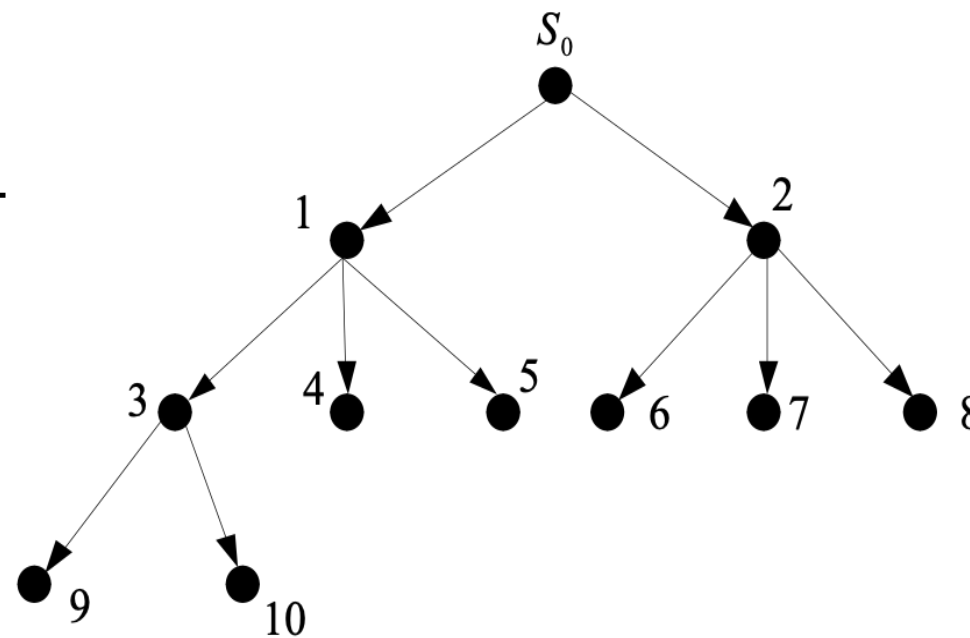


宽度优先搜索

- **宽度优先搜索** (Breadth-first search, 广度优先搜索)：以接近起始节点的程度（深度）为依据，进行逐层扩展的节点搜索方法。

特点：

- (1) 每次选择深度最浅的节点首先扩展，搜索是逐层进行的；
- (2) 一种**高价搜索**，但**若有解存在**，则**必能找到它**。



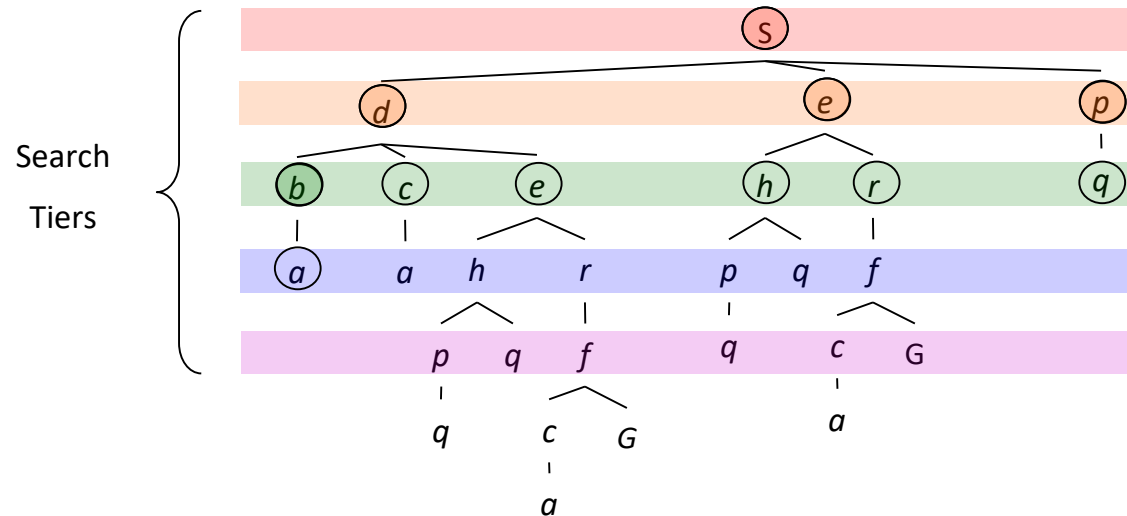
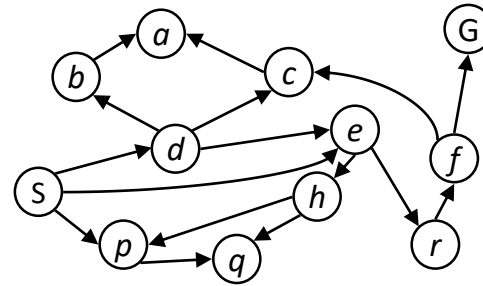
宽度优先搜索法中状态的搜索次序

Breadth-First Search

Strategy: expand a shallowest node first

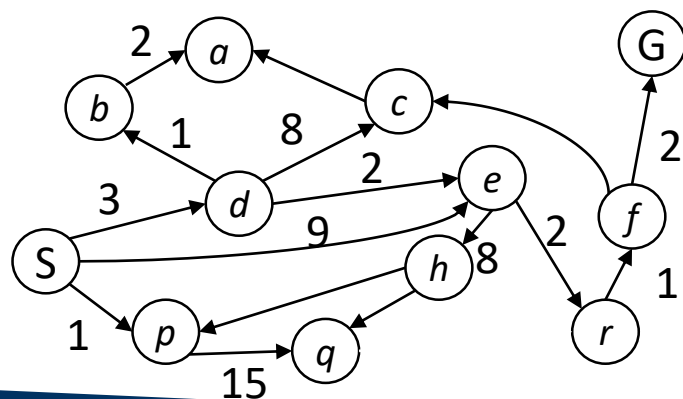
Implementation:

Frontier is a FIFO queue



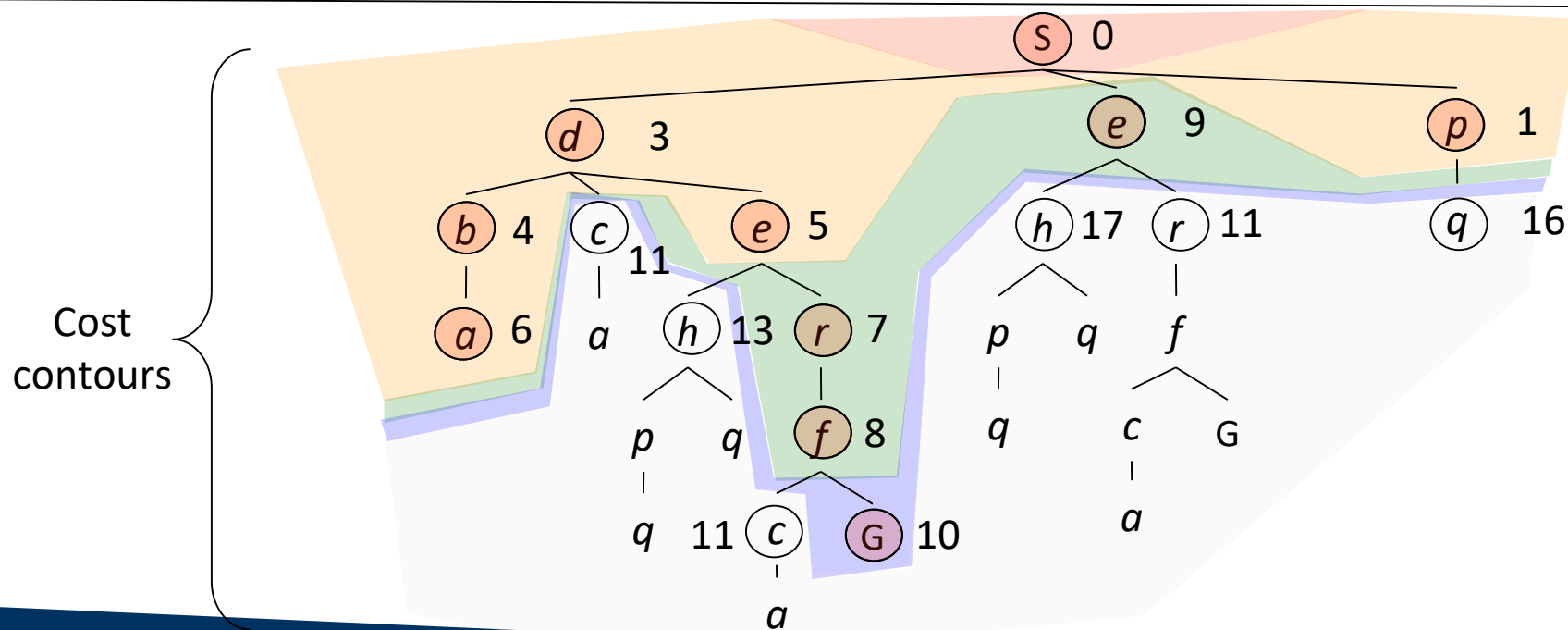
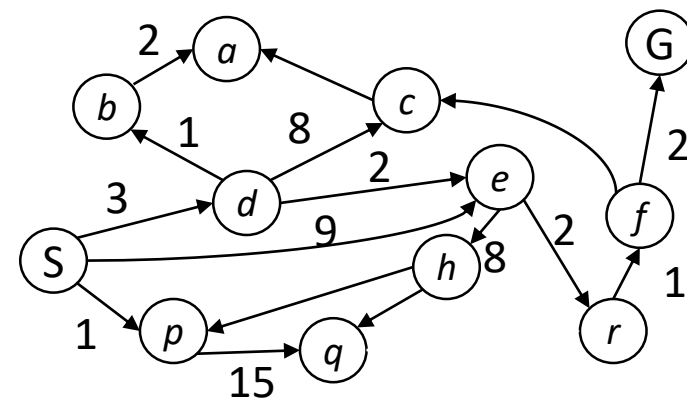
统一代价搜索

- 宽度优先搜索只能处理**无权重图搜索**问题
- 统一代价搜索 (Uniform cost search, UCS) 是对宽度优先的改进, 可以解决**有权重图搜索**问题
- 节点的扩展选择以 $g(n)$ =**从根节点到节点 n 的代价**为依据, 选择 $g(n)$ 最小的节点扩展

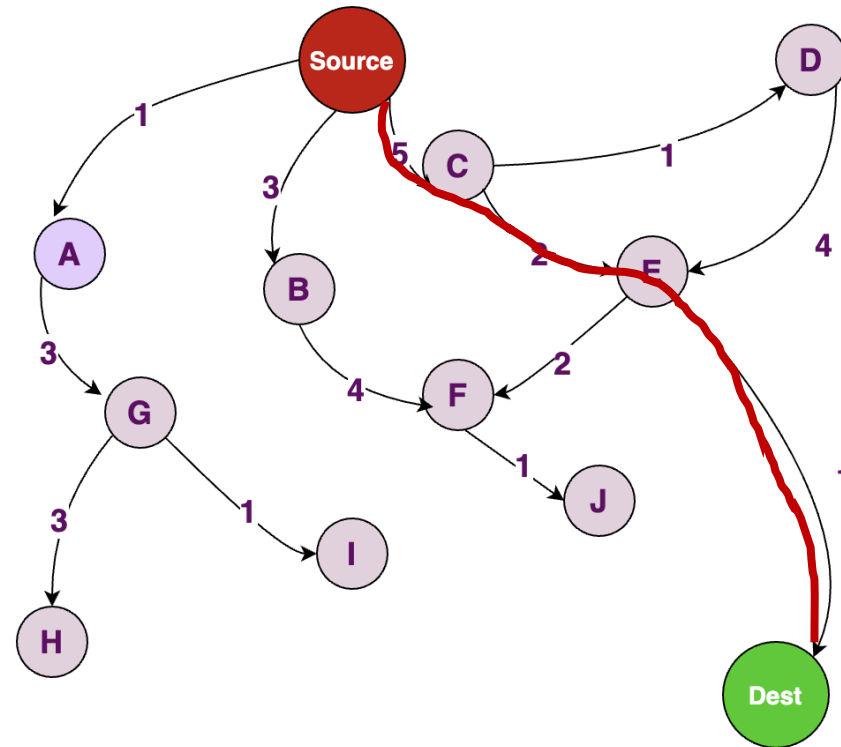


统一代价搜索

$g(n)$ =从根节点到节点n的代价



统一代价搜索-练习

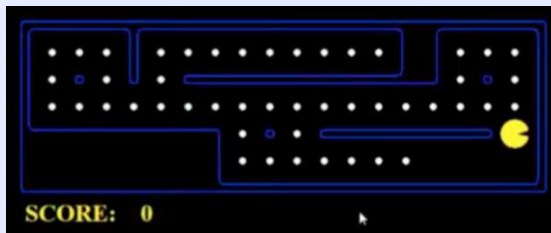


The minimum distance between the source and destination nodes is 8.

	BFS	UCS Dijkstra
Main Concept	Visit nodes level by level based on the closest to the source	In each step, visit the node with the lowest cost
Optimality	Gives an optimal solution for unweighted graphs or weighted ones with equal weights	Gives an optimal solution for both weighted and unweighted graphs
Queue Type	Simple queue	Priority queue
Time Complexity	$O(V + E)$	$O(V + E(\log V))$

小测验：如何建立问题？

环境状态 包含了所有环境中的细节信息

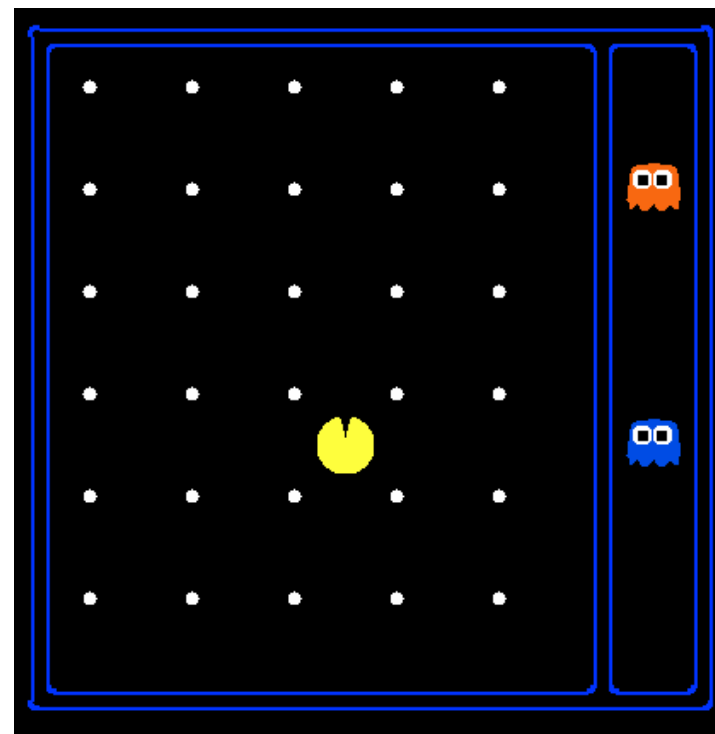


搜索状态 只保留了进行规划所需的信息 (对环境的抽象)

- Problem: Pathing
 - States: (x,y) location
 - Actions: NSEW
 - Transition: update x,y value
 - Goal test: is $(x,y)=\text{destination}$
- Problem: Eat-All-Dots
 - States: $\{(x,y), \text{dot Booleans}\}$
 - Actions: NSEW
 - Transition: update x,y and possibly a dot Boolean
 - Goal test: dots all false

状态空间大小

- 环境状态World state:
 - Agent positions: 120
 - Food count: 30
 - Ghost positions: 12
 - Agent facing: NSEW
- 状态维数:
 - 环境状态?
 $120 \times (2^{30}) \times (12^2) \times 4$
 - 路径规划所需状态?
120
 - 吃光所有豆豆所需的状态?
 $120 \times (2^{30})$



本章主要内容

- 搜索的基本概念
- 状态空间表示方法
- 盲目搜索策略
- 启发式搜索策略

启发式策略

- **启发式信息**：用来简化搜索过程有关具体问题领域的特性信息叫做启发信息
- 启发式图搜索策略（利用启发信息的搜索方法）的特点：重排frontier，选择最有希望的节点加以扩展
- 适合用启发式策略的两种基本情况：
 - (1) 由于**存在问题陈述**和**数据获取**的**模糊性**，可能会使代求解问题**没有一个确定的解**；
 - (2) 虽然一个问题可能有确定解，但是其**状态空间特别大**，搜索中生成扩展的状态数会**随着搜索的深度呈指数级增长**。

启发信息和估价函数

- 在具体求解中，能够**利用与该问题有关的信息**来**简化搜索过程**，称此类信息为启发信息。
- **启发式搜索**：利用启发信息的搜索过程。
- 按运用的方法分类：
 - (1) 陈述性启发信息：用于更准确、更精炼地描述状态
 - (2) 过程性启发信息：用于构造操作算子
 - (3) **控制性启发信息**：表示控制策略的知识
- 按作用分类：
 - (1) **用于扩展节点的选择**，即用于决定应先扩展哪一个节点，以免盲目扩展
 - (2) 用于生成节点的选择，即用于决定要生成哪些后继节点，以免盲目生成过多无用的节点
 - (3) 用于删除节点的选择，即用于决定删除哪些无用节点，以免造成进一步的时空浪费

启发信息和估价函数

- 估价函数 (evaluation function) : 估算节点"希望"程度的量度。
- 估价函数值 $f(n)$: 从初始节点经过 n 节点到达目标节点的路径的最小代价估计值, 其一般形式是

$$f(n) = g(n) + h(n)$$

- $g(n)$: 从初始节点 s_0 到节点 n 的实际代价 ;
- $h(n)$: 从节点 n 到目标节点 s_g 的最优路径的估计代价, 称为启发函数。

$h(n)$ 比重大: 降低搜索工作量, 但可能导致找不到最优解;

$h(n)$ 比重小: 一般导致工作量加大, 极限情况下变为盲目搜索, 但可能可以找到最优解。

启发信息和估价函数

例 八数码问题的启发函数：

- 启发函数1：取一棋局与目标棋局相比，其位置不符的**数码数目**，例如 $h(S_0) = 5$ ；
- 启发函数2：各数码移到目标位置所需移动的**距离的总和**，例如 $h(S_0) = 6$ ；
- 启发函数3：对每一对逆转数码乘以一个倍数，例如3倍，则 $h(S_0) = 3$ ；
- 启发函数4：将位置不符数码数目的总和与3倍数码逆转数目相加，例如 $h(S_0) = 8$ 。

2	1	3
7	6	4
	8	5

初始棋局



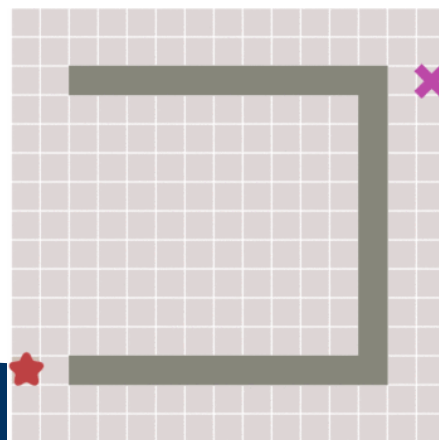
1	2	3
8		4
7	6	5

目标棋局

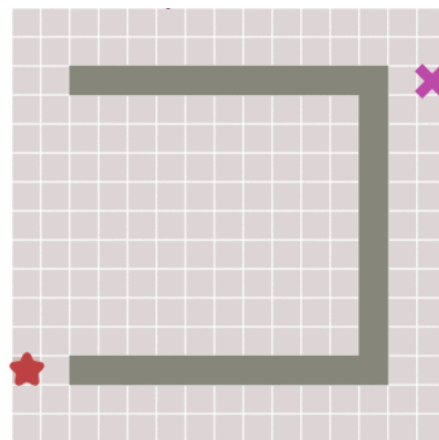
最佳优先搜索

- 最佳优先搜索算法 (Best-First-Search) 是一种启发式搜索算法，基于宽度优先搜索，用启发估价函数对将要被遍历到的点进行估价，然后**选择代价小的进行遍历**（估价函数 $f(n)=h(n)$ ），直到找到目标节点或者遍历完所有点，算法结束。
- BFS算法**不能保证**找到的路径是**一条最短路径**，但是其计算过程相对于UCS算法会更快。

Dijkstra



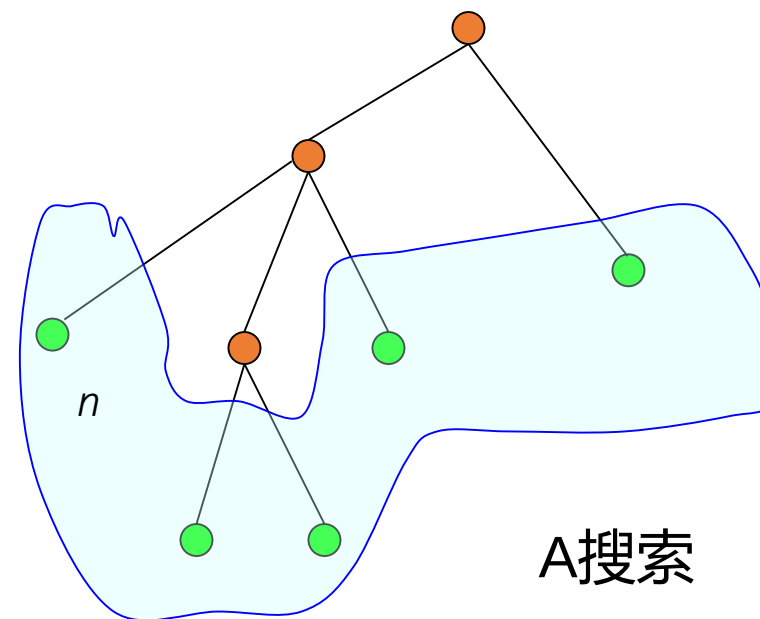
BFS



A*搜索

- A*搜索算法：使用了估价函数 f 的最佳优先搜索。
- 估价函数 $f(n) = g(n) + h(n)$
- 如何寻找并设计启发函数 $h(n)$ ，然后以 $f(n)$ 的大小来排列 OPEN 表中待扩展状态的次序，每次选择 $f(n)$ 值最小者进行扩展。

$g(n)$ ：状态 n 的实际代价，例如搜索的深度；
 $h(n)$ ：对状态 n 与目标“接近程度”的某种启发式估计。



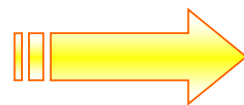
A*搜索

例：利用A搜索算法求解八数码问题，问最少移动多少次就可达到目标状态？

- 估价函数定义为 $f(n) = g(n) + h(n)$
- $g(n)$ ：节点 n 的深度，如 $g(S_0)=0$ 。
- $h(n)$ ：节点 n 与目标棋局不相同的位数（包括空格），简称“不在位数”，如 $h(S_0)=5$ 。

2	8	3
1	6	4
7		5

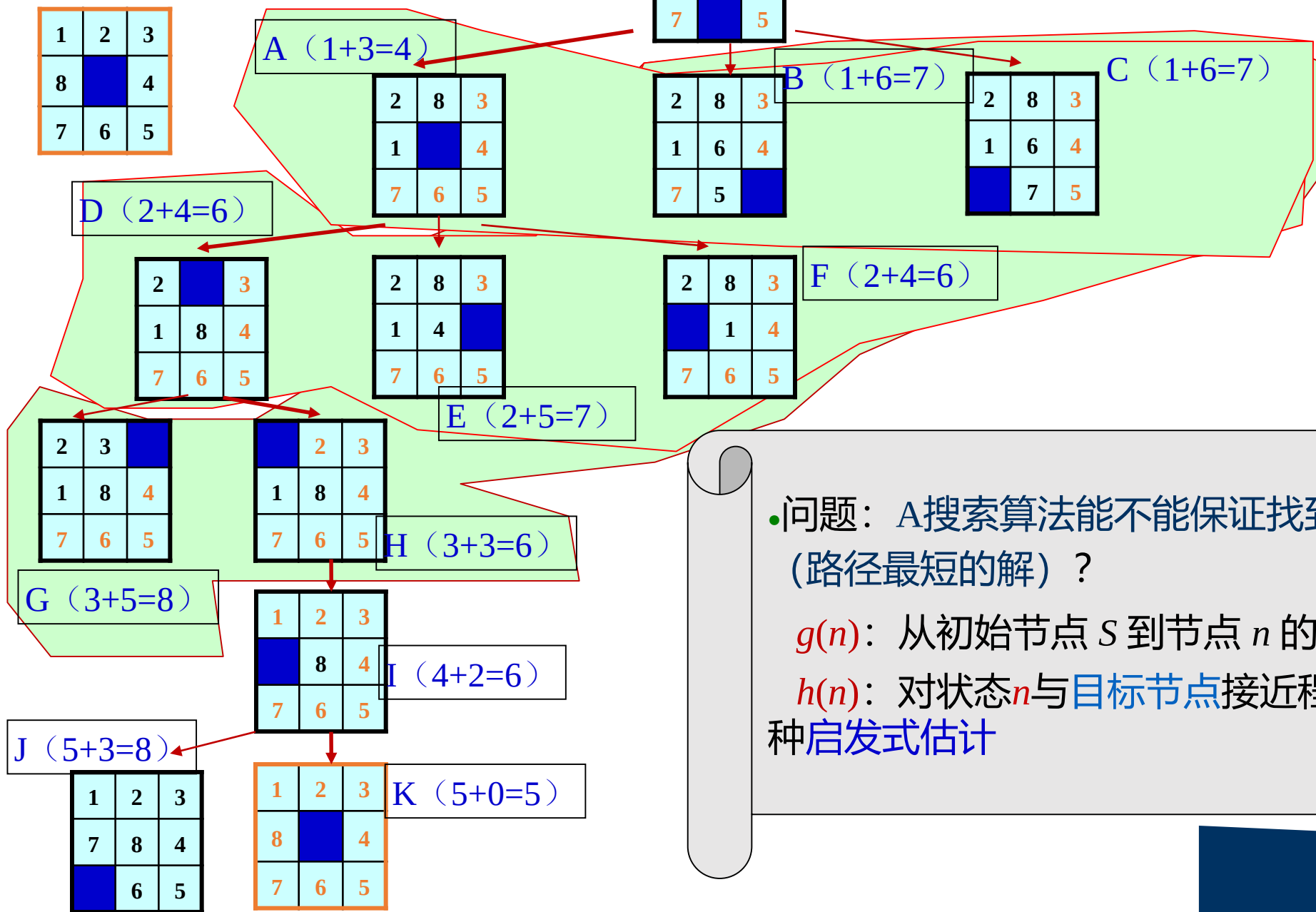
初始状态 S_0



1	2	3
8		4
7	6	5

目标状态

操作算子集: $\uparrow, \downarrow, \rightarrow, \leftarrow$



问题: A搜索算法能不能保证找到最优解 (路径最短的解)?

$g(n)$: 从初始节点 S 到节点 n 的实际代价

$h(n)$: 对状态 n 与目标节点接近程度的某种启发式估计

A*搜索

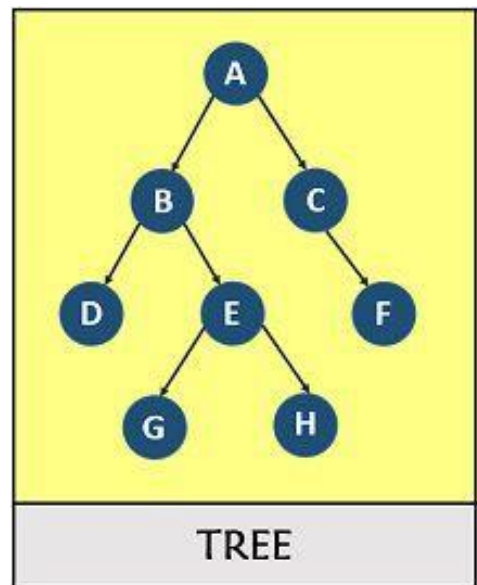
A*搜索算法的特点:

- A*是完备的
- A*是最优的, 如果下列条件成立:

A*搜索的最优性条件

- 启发函数的可采纳性 (Admissibility):

$$h(n) \leq h^*(n)$$



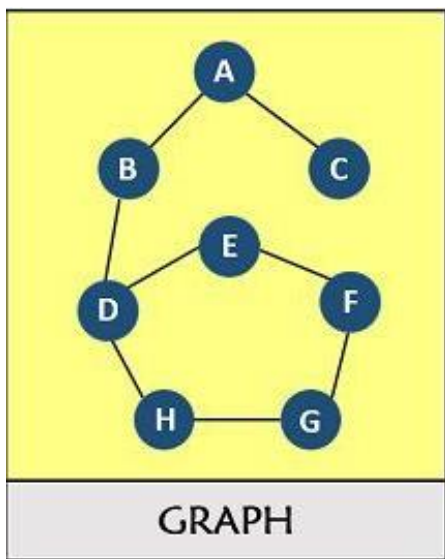
对树搜索总是成立

上例中的八数码问题中定义的 $h(n)$ 表示了“不在位”的数码数，满足上述条件，因此所得解路径为最优解路径。

A*搜索的最优性条件

- 启发函数的单调性 (Consistency / Monotonicity) :

$$h(n) \leq c(n, a, n') + h(n')$$



$c(n, a, n')$ 是从节点 n 到节点 n' 的实际代价。

图搜索中A*最优性的条件

一般来说，大多数可采纳的启发式函数都是单调的。

补充-启发函数的信息性

- 信息性:

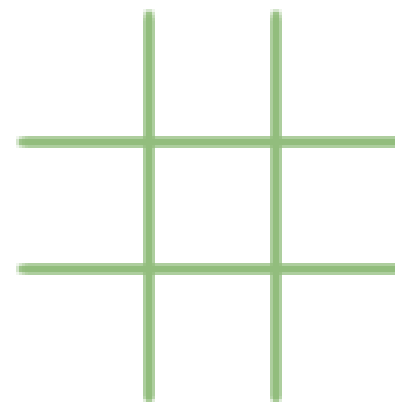
- 在两个A*启发策略的 h_1 和 h_2 中, 如果对搜索空间中的任一状态 n 都有 $h_1(n) \leq h_2(n)$, 就称策略 h_2 比 h_1 具有更多的信息性。
- 如果某一搜索策略的 $h(n)$ 越大, 则A*算法搜索的信息性越多, 所搜索的状态越少。
- 但更多的信息性可能需要更多的计算时间, 也有可能抵消减少搜索空间所带来的益处。

博弈中的搜索

- 例：一字棋

在九宫棋盘上，从空棋盘开始，双方轮流在棋盘上摆各自的棋子×或✕(每次一枚)，谁先取得三子一线（一行、一列或一条对角线）的结果就取胜。

- × 和 ✕能够在棋盘上摆成的各种不同的棋局就是问题空间中的不同状态。
- 9个位置上摆放{空, ×, ✕}有 3^9 种棋局。
- 可能的走法： $9 \times 8 \times 7 \times \dots \times 1$ 。

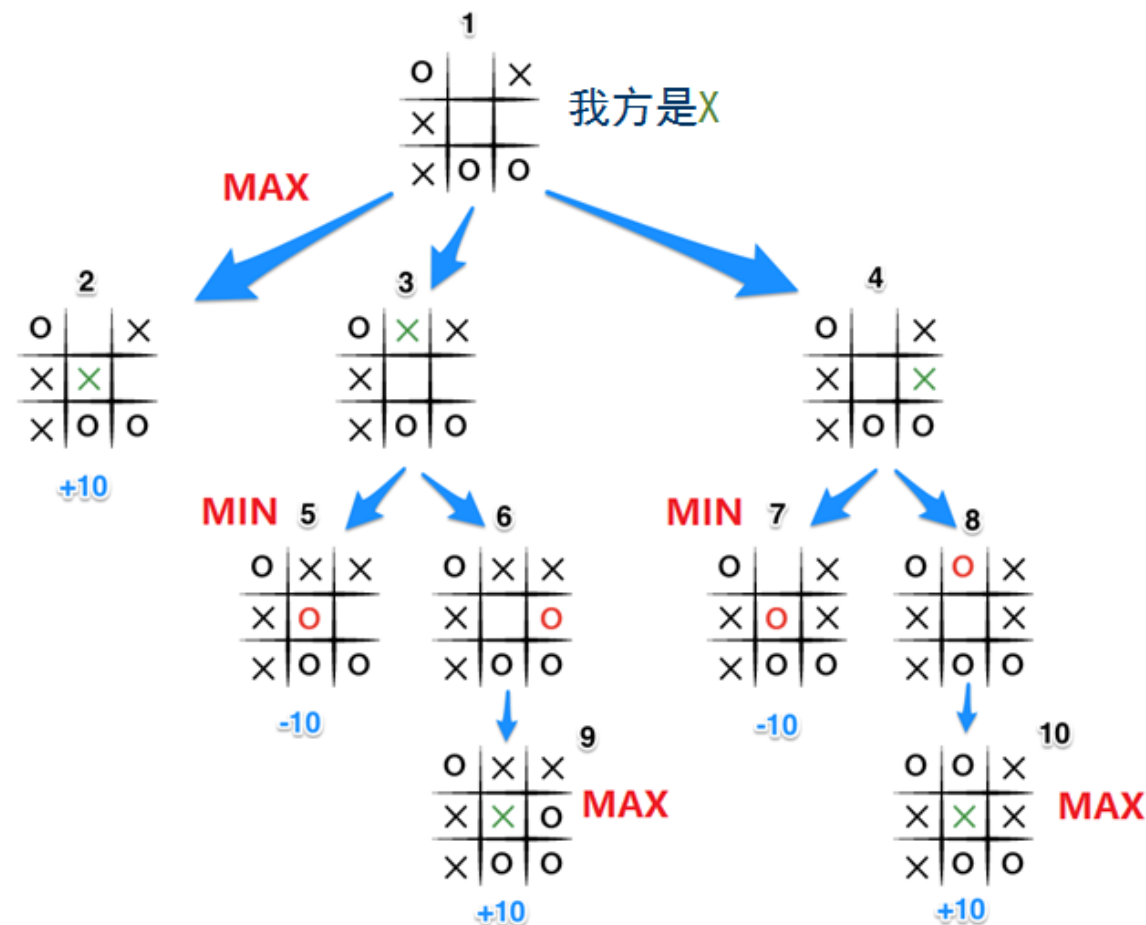
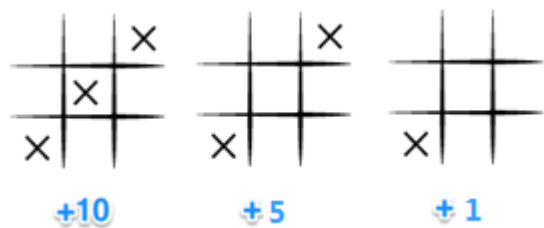


博弈中的搜索

- 博弈树：深度优先遍历下子情况
- MIN-MAX策略：每种棋盘情况有一个估价函数。当我方(MAX方)行动时，必须走棋使估价函数最大的一步，即MAX；而敌方 (MIN)行动时，必然会走棋使估价函数最小的一步，即MIN。

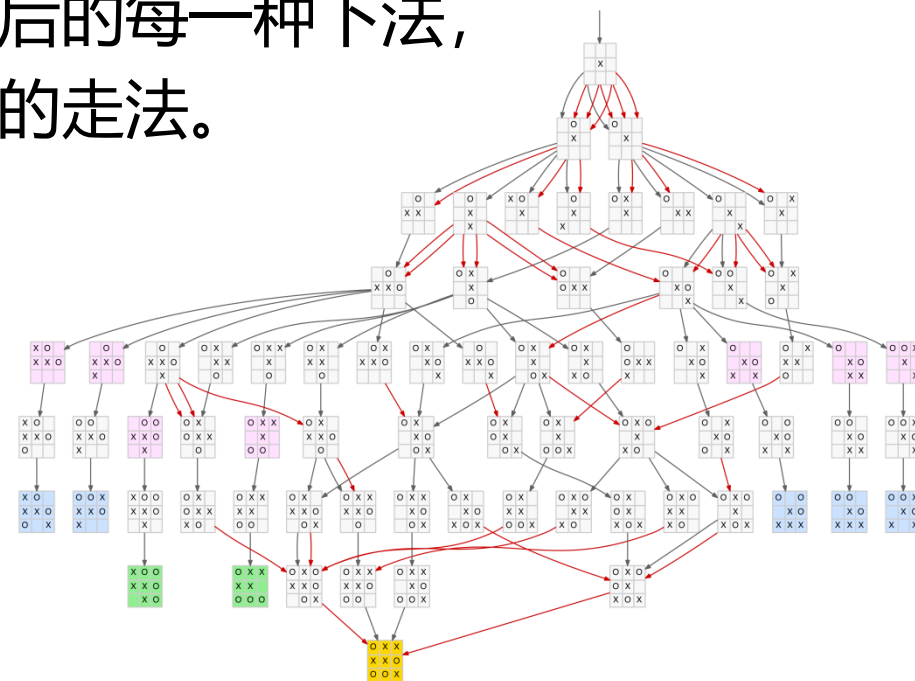
- 设计估价函数：

- 有三个相同的棋子
- 有两个相同的棋子+一个空格
- 有一个相同的棋子+两个空格



博弈中的搜索

- 博弈树搜索时，先假设双方足够聪明，搜索尝试走完所有可能，然后得到行动时如何走棋的步骤。
- 实际是先生成了一整棵博弈树，枚举当前局面后的每一种下法，计算每个局面后续的赢棋概率，选择概率最高的走法。
- 如果搜索树极大，则无法在有效时间内返回结果。
- 使用剪枝算法来减少搜索结点 (α - β 剪枝) 或对结点进行采样 (蒙特卡洛树搜索)



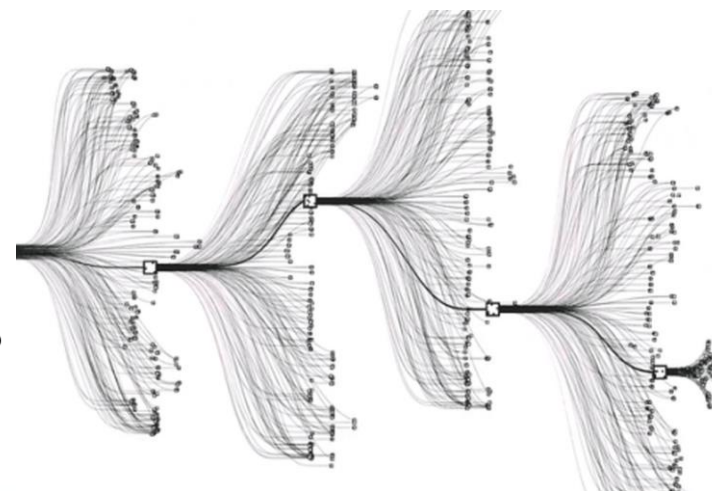
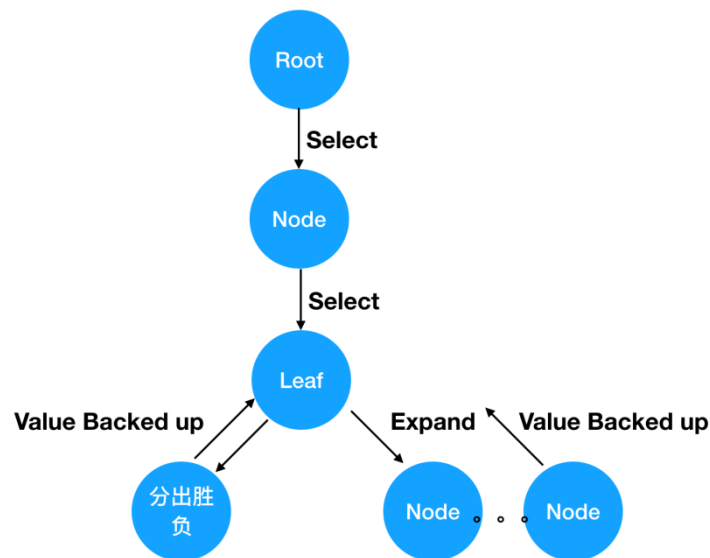
博弈中的搜索

- AlphaGo/Zero
 - 搜索树太广、太深
 - 使用蒙特卡洛树搜索

基本的MCTS有4个步骤：

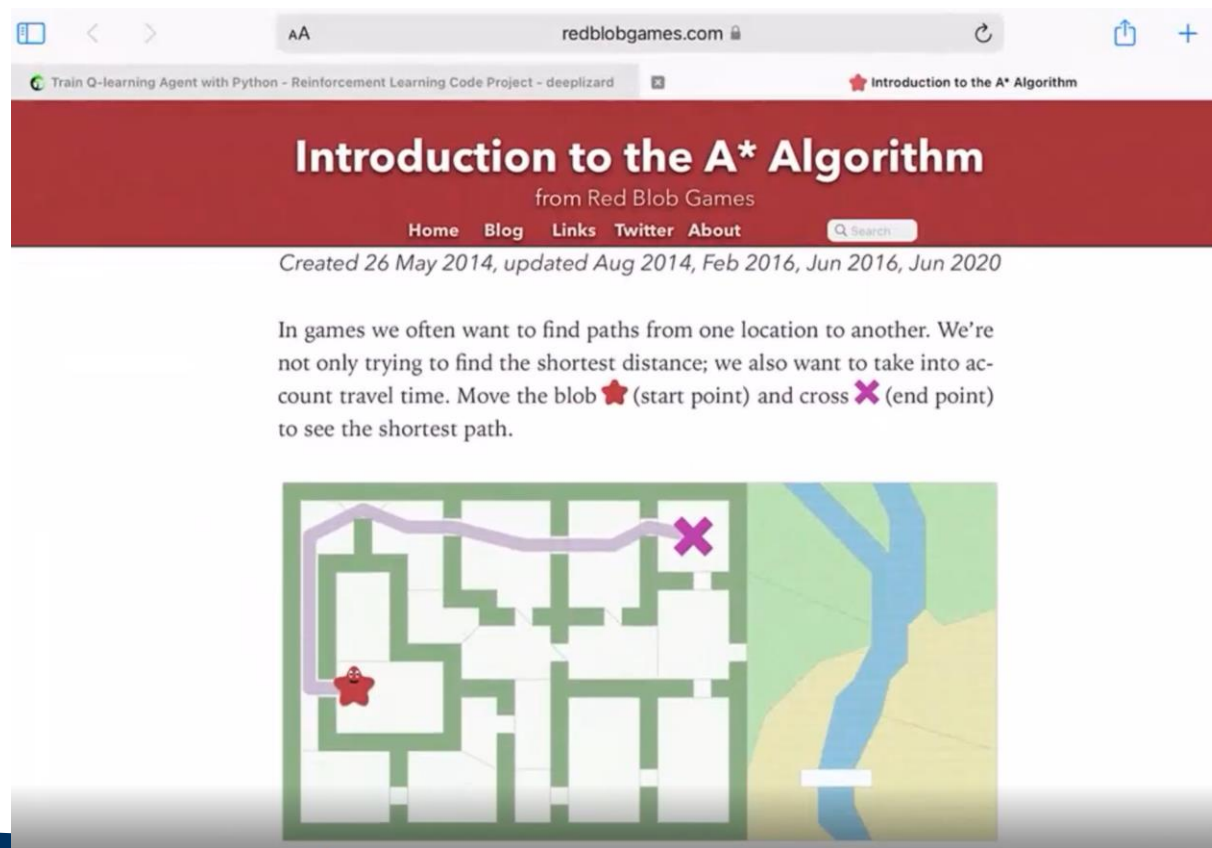
Selection, Expansion,
Simulation, Back-propagation

MCTS：不穷举所有组合，找到最优或次优位置



A*搜索有意思的网站

[Introduction to the A* Algorithm \(redblobgames.com\)](http://redblobgames.com)



第二次作业

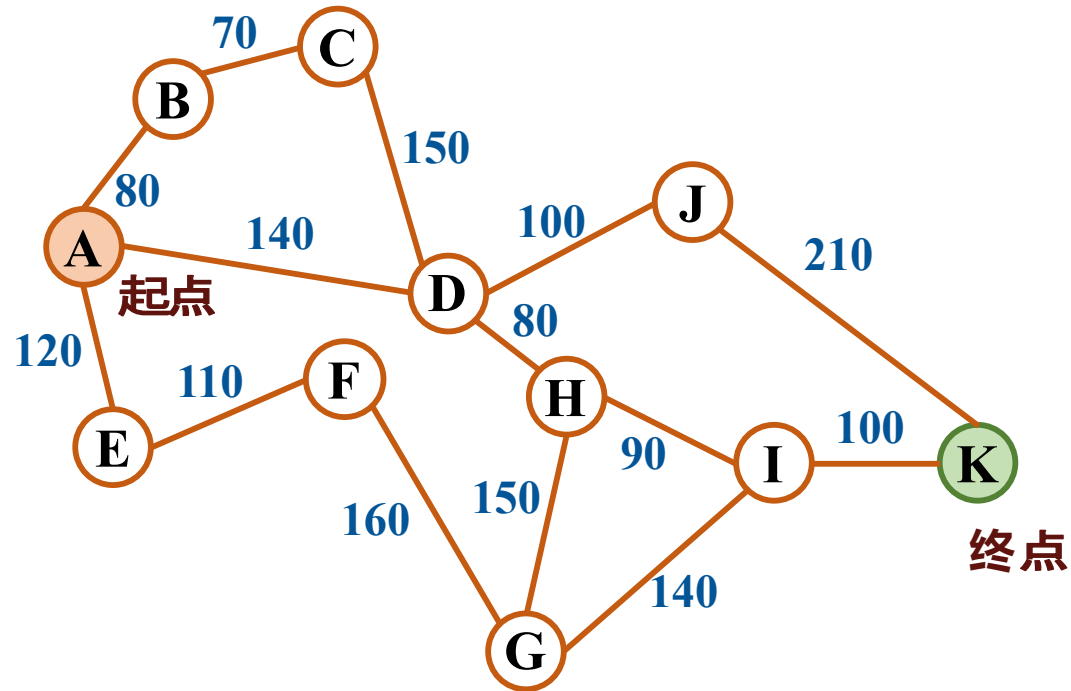
题目1：设有2个修道士和2个野人来到河边，打算乘一条索道从河的左岸渡到河的右岸。但索道只有一个轿箱，每次只能装载1人，在任何岸边野人的数目都不得超过修道士的人数，否则修道士就会被野人吃掉。假设野人和修道士都服从你的过河安排，且索道可自动返回（返回过程无需考虑）。请问如何制定过河计划才能把所有人都安全地渡过河去。请按照深度优先进行求解，列出状态和操作算子，进行相应的符号定义后，画出搜索树，通过搜索树标注具体求解过程。

第二次作业

题目2：某无人车计划从A点开往K点，已知各节点间的实际距离（见下一页的图示），选取启发函数为各节点到终点的直线距离（见下一页的表格）。请利用A*搜索算法找到解路径（给出完整的搜索过程），请问是否为距离最短的路径，从启发函数的特性上解释原因。

第二次作业

题目2:



地点	离终点直线距离
A	370
B	380
C	350
D	250
E	330
F	250
G	240
H	180
I	90
J	170
K	0

第二次作业

截止日期：2026年3月16日